

ЕТАП 10 ОТ 11 · РЪКОВОДСТВО

Полиране الأذان

Езан вече говори и турски: щом телефонът е на турски, приложението, известията, екранът „Аларма“ и widget-ите са на турски – с имената на молитвите от календара на Диянет и традиционните турски имена на градовете в България. Добавена е и достъпност: голям шрифт и TalkBack.

Türkçe

Диянет

Kırcaali · Şumnu · Filibe

Голям шрифт

TalkBack

Jest – 237 теста

14 файла



Какво е решено в този етап

Език по телефона: български → български, турски → турски, другите → английски · молитвите: İmsak, Güneş, Öğle, İkinci, Akşam, Yatsı · името на приложението на турски – „Ezan“ · градовете – с традиционните турски имена, селата – на латиница · търсенето намира и „kırcaali“ · при голям шрифт плътните редове растат до 1,2–1,35 пъти · TalkBack чете деня в таблицата наведнъж · етап 9 (iPhone) е отложен – няма устройство за тест.

СЪДЪРЖАНИЕ

01 Какво прави етапът

02 Нов build

03 Текстовете на турски

04 Градовете на турски

05 Android: widget-и и аларма

06 Достъпност

07 Тестове

08 Проверка на телефона

Стъпка 1

Какво прави етапът

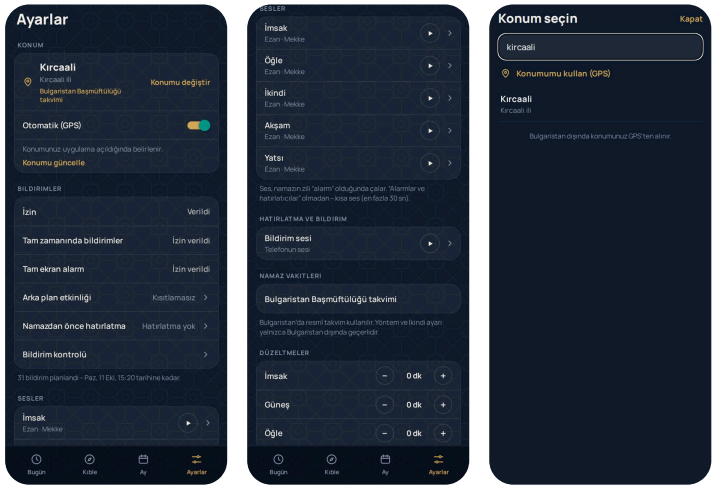
Езикът идва от телефона, както досега – без настройка в приложението:

ЕЗИКЪТ НА ТЕЛЕФОНА	ПРИЛОЖЕНИЕТО
Български	български
Türkçe	турски (нов)
Всеки друг	английски

На Android 13+ езикът може да се избере само за Езан: Settings → Apps → Adhan → Language.



Bugün · Ay · Kible



Ayarlar · Sesler · Konum seçin („kircaali“)

Стъпка 2

Нов build

Има нови ресурси за Android (името на приложението и widget-ите на турски) и малка промяна в Kotlin (датата на екрана „Аларма“) – трябва prebuild и нов build.

```
npx expo prebuild --platform android --clean
npx expo run:android
```

За проба: Settings → Apps → Adhan → Language → Türkçe (или езикът на целия телефон).

Стъпка 3

Текстовете на турски

Всички текстове – в отделен файл, със същата форма като българските (TypeScript дава грешка, ако липсва ключ).

tr.ts src/i18n

- Молитвите – както в календара на Диянет: **İmsak, Güneş, Öğle, İkindi, Akşam, Yatsı**; месеците по Хиджра – Muharrem ... Ramazan ... Zilhicce.
- В турския окончанието зависи от думата (ikindi'ye, Akşam'a, Yatsı'ya). Затова изреченията с името на молитвата са построени с „vakti / vaktine“: „İkindi vakti“, „Akşam vaktine 10 dk“ – винаги правилно.
- Настройките на телефона (Samsung, Xiaomi, Huawei...) – с имената им в турския интерфейс („Alarmlar ve hatırlatıcılar“, „Kısıtlanmamış“...).

src/i18n/tr.ts

430 реда

```
import type { Strings } from './strings';

/**
 * Türkçe (етап 10). Имената на молитвите и месеците – както в календара на Диянет
 * (İmsak, Güneş, Öğle, İkindi, Akşam, Yatsı). Където името на молитвата влиза в изречение,
 * след него стои „vakti/vaktine“ – така не трябва турски падежни наставки.
 * Настройките на телефона (Samsung, Xiaomi...) – с имената им в турския интерфейс.
 */
export const tr: Strings = {
  prayers: {
    fajr: 'İmsak',
    sunrise: 'Güneş',
    dhuhr: 'Öğle',
    asr: 'İkindi',
    maghrib: 'Akşam',
    isha: 'Yatsı',
  },
  alert: {
    off: 'bildirim yok',
    notify: 'bildirim',
    adhan: 'alarm',
  },
  tabs: {
    today: 'Bugün',
    qibla: 'Kıble',
    month: 'Ay',
    settings: 'Ayarlar',
  },
  hero: {
```

```

    next: 'Sıradaki',
    at: (time) => `saat ${time}`,
    tomorrowAt: (time) => `yarın ${time}`,
    remaining: (time) => `${time} kaldı`,
  },

  nowPill: 'ŞİMDİ',

  busy: 'Yükleniyor...',
  locating: 'Konumunuz bulunuyor...',
  choosePlace: 'Yer seçin',
  noPlaceBanner: 'Konum bulunamadı - gösterilen vakitler Sofya içindir. Yerinizi seçmek için dokunun.',

  ally: {
    city: (city) => `Şehir: ${city}. Şehri değiştir`,
    locating: 'Konumunuz bulunuyor',
    bell: (prayer, mode) => `${prayer}: ${mode}. Değiştirmek için dokunun`,
  },

  toast: (prayer, mode) => `${prayer}: ${mode}`,
  alarmToast: (prayer) => `${prayer}: alarm - sessiz modda da çalar`,

  placeholder: {
    qibla: 'Kâbe'yi gösteren pusula. 7. aşamada gelecek.',
  },

  month: {
    dayColumn: 'Gün',
    hijriColumn: 'Hicri',
    today: 'Bugün',
    prev: 'Önceki ay',
    next: 'Sonraki ay',
    friday: 'Cuma',
    sourceMufti: 'Kaynak: Bulgaristan Başmüftülüğü',
    sourceCalc: (method) => `${method} yöntemiyle hesaplandı`,
  },

  settings: {
    title: 'Ayarlar',
    groupPlace: 'Konum',
    groupTimes: 'Namaz vakitleri',
    groupCorrections: 'Düzeltilmeler',
    groupApp: 'Uygulama',
    about: 'Hakkında',
    change: 'Konumu değiştir',
    auto: 'Otomatik (GPS)',
    autoHint: 'Konumunuz uygulama açıldığında belirlenir.',
    locating: 'Konumunuz belirleniyor...',
    denied: 'Konum izni kapalı.',
    openSettings: 'Telefon ayarlarını aç',
    unavailable: 'Konumunuz belirlenemedi. Bilinen son yer gösteriliyor.',
    refresh: 'Konumu güncelle',
    sourceMufti: 'Bulgaristan Başmüftülüğü takvimi',
    sourceMuftiNote: 'Bulgaristan'da resmî takvim kullanılır. Yöntem ve İkinci ayarı yalnızca Bulgaristan dışında geçerlidir.',
    sourceCalc: 'Bu yer için hesaplandı',
    method: 'Yöntem',
    methodAuto: 'Otomatik',
    methodAutoValue: (method) => `Otomatik · ${method}`,
    methodAutoDesc: (country, method) => `Ülkeye göre: ${country} → ${method}`,
    asr: 'İkinci',
  },

```

```

shafi: 'Şâfiî, Mâlikî, Hanbelî',
hanafi: 'Hanefî',
shafiDesc: 'Gölge cismin boyu kadar - Diyanet takvimindeki gibi "birinci" İkindi',
hanafiDesc: 'Gölge cismin iki katı - yaklaşık bir saat sonra',
asrNote: 'Şâfiî, Mâlikî ve Hanbelî İkindi'yi aynı hesaplar; yalnızca Hanefî daha geçtir.',
polarNote: (lat) =>
    `${lat}° enlemde kutup gündüzleri ve geceleri vardır. Güneşin doğmadığı veya batmadığı günlerde
    vakitler 65° enleme göre - normal gece ve gündüzün olduğu en yakın yerlere göre - hesaplanır. Bölgenizdeki
    camiye danışın.`,
    highLat: 'Yüksek enlemler',
    highLatNote: (lat) =>
        `${lat}° enlemde yazın gerçek alacakaranlık olmaz; İmsak ve Yatsı bu kurala göre belirlenir.
        Bölgenizdeki camiye danışın.`,
    offsetsNote: 'Caminiz birkaç dakika farklıysa buradan düzeltin. Tüm ekranlarda geçerlidir.',
    minutes: (n) => (n === 0 ? '0 dk' : `${n > 0 ? '+' : ''}${n} dk`),
    hijri: 'Hicri tarih',
    days: (n) => (n === 0 ? '0 gün' : `${n > 0 ? '+' : ''}${n} gün`),
    hijriNote: 'Hicri tarih Bulgaristan Başmüftülüğünün ilan ettiğinden farklıysa.',
    hijriNoteAbroad: 'Hicri tarih bölgenizdeki caminin ilan ettiğinden farklıysa.',
    reset: 'Düzeltilmeleri sıfırla',
    decrease: (label) => `Azalt: ${label}`,
    increase: (label) => `Artır: ${label}`,
},

widget: {
    next: 'Sıradaki',
    tomorrowShort: (time) => `yarın ${time}`,
    until: (name) => `${name} vaktine kalan`,
    left: 'kaldı',
    empty: 'Vakitleri görmek için Ezan'ı açın.',
},

notifications: {
    prayerTitle: (prayer, time) => `${prayer} - ${time}`,
    prayerBody: (prayer, arabic, place) => `${prayer} vakti girdi (${arabic}) • ${place}`,
    sunriseBody: (place) => `Güneş doğdu - İmsak vakti sona erdi • ${place}`,
    nextLine: (prayer, time, tomorrow) => `Sıradaki: ${prayer}, ${tomorrow ? 'yarın ' : ''}saat ${time}`,
    reminderTitle: (prayer, minutes, time) => `${prayer} vaktine ${minutes} dk - ${time}`,
    reminderBody: (place) => `Abdest alıp namaza hazırlanma vakti • ${place}`,
    refreshTitle: 'Ezan'ı açın',
    refreshBody: 'Namaz bildirimlerinin gelmeye devam etmesi için uygulamayı açın.',
    channelPrayer: 'Namazlar',
    channelAdhan: 'Alarm',
    channelReminder: 'Hatırlatmalar',
    group: 'Bildirimler',
    status: 'İzin',
    granted: 'Verildi',
    denied: 'Reddedildi',
    undetermined: 'Henüz sorulmadı',
    allow: 'Bildirimlere izin ver',
    deniedNote: 'Bildirimler telefon ayarlarında kapalı - bunlar olmadan namaz bildirimleri ve alarmlar
    çalışmaz.',
    webNote: 'Bildirimler yalnızca telefon uygulamasında çalışır.',
    reminder: 'Namazdan önce hatırlatma',
    reminderOff: 'Hatırlatma yok',
    reminderValue: (n) => `${n} dk önce`,
    vibrate: 'Titreşim',
    exact: 'Tam zamanında bildirimler',
    exactOff: 'Kapalı',
    exactNote: '"Alarmlar ve hatırlatıcılar" izni olmadan Android bildirimleri birkaç dakika
    geciktirebilir.',

```

```

exactAllow: '"Alarmlar ve hatırlatıcılar"a izin ver',
scheduled: (n, until) => `${n} bildirim planlandı - ${until} tarihine kadar`,
scheduledNone: 'Planlanmış bildirim yok - "Bugün" ekranındaki tüm zil simgeleri kapalı.',
testTitle: 'Deneme bildirim',
testBody: 'Namaz bildirimleri böyle görünecek.',
bellDenied: 'Bildirimlere izin verilmedi - bildirimler ve alarmlar çalışmaz.',
allowedToast: '✓ Bildirimlere izin verildi',
exactBanner: 'Bildirimler birkaç dakika gecikebilir. Tam zamanında bildirimlere izin vermek için dokun.',
deniedBanner: 'Bildirimlere izin verilmedi - bildirimler ve alarmlar çalışmaz. İzin vermek için dokun.',
diagnostics: 'Bildirim kontrolü',
diag: {
  title: 'Bildirim kontrolü',
  subtitle: 'Telefon Ezan bildirimlerini zamanında gösteriyor mu - göstermiyorsa neyin düzeltilmesi gerekir.',
  refresh: 'Yenile',
  updated: (time) => `Güncellendi: ${time}`,
  status: 'Durum',
  permOk: 'Bildirimlere izin verildi',
  permBad: 'Bildirimlere izin verilmedi',
  exactOk: 'Tam zamanında bildirimler ("Alarmlar ve hatırlatıcılar")',
  exactBad: 'Tam zamanında bildirimler kapalı - gecikebilir',
  allow: 'İzin ver',
  scheduledOk: (n, until) => `${n} bildirim planlandı · ${until} tarihine kadar`,
  scheduledNone: 'Planlanmış bildirim yok - "Bugün" ekranındaki zil simgeleri kapalı',
  test: 'Deneme bildirim',
  testNotification: 'Bildirim',
  testIn: (s) => `${s} sn sonra`,
  testHint: '10 sn sonra gelir. Gerçekte nasıl geleceğini görmek ve duymak için telefonu kilitleyin.
Alarm sesi Ayarlar → Sesler bölümünden dinlenebilir.',
  recent: 'Son bildirimler',
  recentNone: 'Henüz Ezan bildirim gösterilmedi.',
  onTime: 'zamanında',
  late: (s) => (s < 60 ? `+${s} sn` : `+${Math.round(s / 6) / 10} dk`),
  plannedShown: (planned, shown) => `planlanan ${planned} · gösterilen ${shown}`,
  upcoming: 'Yaklaşan bildirimler',
  kindAlarm: 'Alarm',
  kindNotification: 'Bildirim',
  kindReminder: 'Hatırlatma',
  kindOther: 'Diğer',
  tech: 'Teknik ayrıntılar',
  device: 'Telefon',
  channelLine: (importance, sound, vibrate, usage) =>
    `önem ${importance} · ses ${sound} · titreşim ${vibrate ? 'evet' : 'hayır'} · akış ${usage}`,
  reset: 'Tüm bildirimleri yeniden planla',
  resetHint: 'Tüm Ezan bildirimlerini siler ve geçerli ayarlarla yeniden planlar. Bir şey yanlış görünüyorsa kullanın.',
  resetDone: '✓ Tüm bildirimler yeniden planlandı.',
  fullScreenOk: 'Tam ekran alarm',
  fullScreenBad: 'Tam ekran alarm kapalı',
  batteryOk: 'Arka planda kısıtlamasız çalışıyor',
  batteryBad: 'Telefon arka plan etkinliğini kısıtlıyor',
  fix: 'Düzelt',
},
fullScreen: 'Tam ekran alarm',
fullScreenOn: 'İzin verildi',
fullScreenOff: 'Kapalı',
fullScreenNote: 'Bu olmadan ezan kilitli telefonda yine çalar, ancak tam ekran yerine yalnızca bir bildirim görürsünüz.',
fullScreenAllow: '"Tam ekran alarm"a izin ver',

```

```

exactOn: 'İzin verildi',
background: 'Arka plan etkinliği',
backgroundOk: 'Kısıtlamasız',
backgroundBad: 'Kısıtlı',
batteryTitle: 'Alarmlar gecikebilir',
batteryText: 'Telefon pil tasarrufu için uygulamayı durdurabilir.',
batteryFix: 'Düzeltil',
},

qibla: {
  title: 'Kible',
  fromNorth: 'kuzeyden',
  toMecca: 'Mekke'ye',
  km: 'km',
  aligned: '✓ Kibleye dönüktünüz',
  turnLabel: (direction) => (direction === 'right' ? 'Sağa dönün' : 'Sola dönün'),
  starting: 'Pusulayı başlatılıyor...',
  quality: { high: 'yüksek', medium: 'orta', low: 'düşük' },
  accuracy: (q) => `Pusulayı: ${q} doğruluk`,
  tip: 'Telefonu ekranı yukarı bakacak şekilde düz tutun. Yakındaki metal ve mıknatıslar pusulayı
şaşırtır.',
  calibrateTitle: 'Pusulayı kalibre edin',
  calibrateText: 'Telefonu havada 2-3 kez sekiz çizerek hareket ettirin. Metalden, mıknatıslardan ve
mıknatıslı kılıflardan uzak tutun.',
  noCompassTitle: 'Pusulayı yok',
  noCompassText: (deg) =>
    `Bu telefonda manyetik sensör yok. Kible kuzeyden ${deg}°: kuzeye dönün ve ${deg}° sağa dönün.`,
  permissionTitle: 'İzin gerekli',
  permissionText: 'iPhone\'da pusula yalnızca konum izniyle çalışır.',
  allowLocation: 'Konuma izin ver',
  letters: ['K', 'D', 'G', 'B'],
  allyDial: (bearing) => `Pusulayı. Kible kuzeyden ${bearing}°`,
  sunToday: 'Güneş bugün tam Kible yönünde: saat {t}',
  sunTomorrow: 'Güneş yarın tam Kible yönünde olacak: saat {t}',
  sunNow: 'Şu anda güneş tam Kible yönünde',
  sunrise: (azimuth, degrees, side) =>
    side === 'same'
      ? `Gün doğumu ${azimuth}° - tam Kible yönünde`
      : `Gün doğumu ${azimuth}° - Kible, gün doğumunun ${degrees}° ${side === 'right' ? 'sağında' :
'solunda'}`,
},

sounds: {
  group: 'Sesler',
  notifyGroup: 'Hatırlatma ve bildirim',
  notifyRow: 'Bildirim sesi',
  noteAndroid: 'Ses, namazın zili "alarm" olduğunda çalar. "Alarmlar ve hatırlatıcılar" olmadan - kısa
ses (en fazla 30 sn).',
  noteIos: 'Ses, namazın zili "alarm" olduğunda çalar. iPhone\'da ses en fazla 30 sn\'dir.',
  fullOffTitle: 'Tam ezan kapalı',
  fullOffText: '"Alarmlar ve hatırlatıcılar" olmadan yalnızca kısa ses çalar (en fazla 30 sn). Tam ezan
için "Alarmlar ve hatırlatıcılar"a izin verin.',
  allow: 'İzin ver',
  title: (prayer) => `${prayer} sesi`,
  notifyTitle: 'Bildirim sesi',
  fullSection: 'Tam ses - alarm',
  fullHint: 'Sonuna kadar çalar; "Durdur" keser.',
  shortSection: 'Kısa ses - en fazla 30 sn',
  shortHintAndroid: '"Alarmlar ve hatırlatıcılar" kapalıyken. Kendi sesiniz bir duraklamada, yumuşak bir
kısılmayla 30 sn\'ye kısaltılır.',
  iosSection: 'Ses - en fazla 30 sn',

```

```

    iosHint: 'iPhone’da bildirim sesi en fazla 30 sn olabilir. Daha uzun kayıtlar bir duraklamada, yumuşak
    bir kısılmayla kısaltılır.',
    notifyHint: 'Bildirimler ve namazdan önceki hatırlatmalar için.',
    byDefault: 'varsayılan',
    mine: 'BENİM',
    add: 'Kendi sesini ekle',
    addFullHint: 'mp3, m4a, mp4, wav, ogg, flac · telefondan',
    addShortHint: 'mp3, m4a, mp4, wav... · uzunsa 30 sn’ye kısaltılır',
    addNotifyHint: 'en fazla 10 sn · mp3, m4a, mp4, wav...',
    tooLongNotify: (name, dur) => `“${name}” ${dur} uzunluğunda - bildirim sesi en fazla 10 sn olabilir.
    Daha kısa bir ses seçin.`,
    adding: 'Ekleniyor...',
    workingShort: 'Kesit kesiliyor...',
    workingFull: 'Ses ekleniyor...',
    workingHint: '30. saniyeden önceki duraklama aranıyor. Birkaç saniye sürebilir.',
    applyAll: 'Tüm namazlar için aynı ses',
    apply: 'Uygula',
    applied: '✓ Tüm namazlara uygulandı',
    iosFuture: 'iPhone’da tam ezan AlarmKit (iOS 26) ile gelecek - 9. aşama.',
    added: (name) => `✓ Eklendi: ${name}`,
    addedTrimmed: (name, dur) =>
        `✓ Eklendi: ${name}. Kısa ses en fazla 30 sn’dir - ilk ${dur} bir duraklamaya kadar, yumuşak bir
        kısılmayla çalar.`,
    trimmed: (dur, of) => `${of} içinden ilk ${dur} çalar`,
    tooLongFull: (name) => `“${name}” 15 dakikadan uzun.`,
    tooBig: 'Dosya 60 MB’tan büyük.',
    unreadable: 'Bu dosya ses olarak okunamıyor.',
    unsupported: 'Kendi seslerinizi yalnızca telefon uygulamasında ekleyebilirsiniz.',
    deleteTitle: (name) => `“${name}” kaldırılsın mı?`,
    deleteText: 'Bu sesi kullanan namazlar varsayılan sese döner.',
    delete: 'Kaldır',
    cancel: 'Vazgeç',
    play: (name) => `${name} - çal`,
    stop: (name) => `${name} - durdur`,
},

alarm: {
    app: 'Ezan',
    title: (prayer) => `${prayer} vakti`,
    stop: 'Ezanı durdur',
    stopAlarm: 'Alarmı durdur',
    mute: 'Sesi kapat',
    muteShort: 'Sessiz',
    close: 'Kapat',
    soundName: 'Ezan',
    channel: 'Namaz alarmı',
},

battery: {
    title: 'Arka plan etkinliği',
    lede: 'Telefon pil tasarrufu için Ezan’ı uyutabilir; alarm gecikebilir ya da hiç çalmayabilir.
    Uygulamanın kısıtlamasız çalışmasına izin verin.',
    yourPhone: 'Telefonunuz',
    open: 'Ezan ayarlarını aç',
    openList: '“Pil optimizasyonu” listesi',
    now: 'Şu an',
    ok: 'Kısıtlamasız ✓',
    bad: 'Kısıtlı',
    unknown: 'kontrol edilemiyor',
    otherPhone: 'Başka bir telefon mu?',
    more: 'Her marka için daha fazla ipucu: dontkillmyapp.com',
}

```

```

brands: {
  samsung: 'Samsung',
  xiaomi: 'Xiaomi / Redmi / POCO',
  huawei: 'Huawei / Honor',
  oppo: 'OPPO / Realme / OnePlus',
  other: 'Diğer',
},
steps: {
  samsung: [
    ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
    ['Açın:', 'Pil'],
    ['Seçin:', 'Kısıtlanmamış', '"Optimize edildi" ya da "Kısıtlanmış" değil.'],
    ['İsteğe bağlı:', 'Ayarlar → Pil → Arka plan kullanım sınırları', 'Ezan "Derin uykudaki uygulamalar" veya "Uykudaki uygulamalar" içinde olmamalı.'],
  ],
  xiaomi: [
    ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
    ['Açın:', 'Pil tasarrufu', '"Kısıtlama yok" seçeneğini seçin.'],
    ['Açın:', 'Otomatik başlatma'],
    ['İsteğe bağlı: son uygulamalarda Ezan kartını basılı tutun ve kilitleyin (asma kilit).'],
  ],
  huawei: [
    ['Açın:', 'Ayarlar → Pil → Uygulama başlatma'],
    ['Ezan'ı bulun ve kapatın:', 'Otomatik olarak yönet'],
    ['Açılan pencerede üçünü de açın:', 'Otomatik başlatma · İkincil başlatma · Arka planda çalıştır'],
  ],
  oppo: [
    ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
    ['Açın:', 'Pil kullanımı'],
    ['Açın:', 'Arka plan etkinliğine izin ver', 've varsa "Otomatik başlatma"yı.'],
  ],
  other: [
    ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
    ['Açın:', 'Pil'],
    ['Seçin:', 'Kısıtlanmamış', 'Bazı telefonlarda adı "Optimize etme"dir.'],
  ],
},
},

about: {
  title: 'Hakkında',
  name: 'Ezan',
  back: 'Ayarlar',
  version: (v: string) => `Sürüm ${v}`,
  tagline: 'Namaz vakitleri, hatırlatmalar ve ezan',
  groupTimes: 'Vakitlerin kaynakları',
  groupPlaces: 'Yer verileri',
  groupPrivacy: 'Gizlilik',
  sources: {
    mufti: { name: 'Bulgaristan Başmüftülüğü', desc: 'Namaz takvimi - Bulgaristan'daki vakitler' },
    adhan: { name: 'adhan', desc: 'Bulgaristan dışında hesaplama · MIT lisansı' },
    osm: { name: 'OpenStreetMap', desc: 'Bulgaristan'daki yerler · © OpenStreetMap katkıcıları, ODbL' },
    ne: { name: 'Natural Earth', desc: 'Bulgaristan sınırı · kamu malı' },
    geonames: { name: 'GeoNames', desc: 'Dünya şehirleri - internetsiz ad ve ülke · CC BY 4.0' },
    praytimes: { name: 'PrayTimes.org', desc: 'Ezan kayıtları: Mekke, Medine, Mescid-i Aksa' },
  },
  privacy:
    'Konumunuz yalnızca telefonda, yeri ve vakitleri seçmek için kullanılır. Uygulama onu hiçbir yere göndermez. Bulgaristan dışında telefon şehrin adını kendi harita hizmetine sorar.',
},

```

```

methods: {
  Turkey: 'Diyanet (Türkiye)',
  MuslimWorldLeague: 'Müslüman Dünya Birliği',
  MoonsightingCommittee: 'Moonsighting Committee',
  Egyptian: 'Mısır Genel Araştırma Kurumu',
  Karachi: 'Karaçi Üniversitesi',
  UmmAlQura: 'Ümmü'l-Kurâ (Mekke)',
  NorthAmerica: 'ISNA (Kuzey Amerika)',
  Dubai: 'Dubai (BAE)',
  Kuwait: 'Kuveyt',
  Qatar: 'Katar',
  Singapore: 'Singapur (MUIS)',
  Tehran: 'Tahran (Jeofizik Enstitüsü)',
  France: 'Fransa (UOIF, 12°)',
  Russia: 'Rusya (DUM)',
  Malaysia: 'Malezya (JAKIM)',
  Indonesia: 'Endonezya (Kemenag)',
},

highLatRules: {
  seventhofthenight: { name: 'Gecenin yedide biri', desc: 'Avrupa'da en yaygın olanı' },
  middleofthenight: { name: 'Gecenin yarısı', desc: 'İmsak ve Yatsı gece yarısını geçmez' },
  twilightangle: { name: 'Alacakaranlık açısı', desc: 'Yöntemin açısına göre gecenin bir payı' },
},

place: {
  title: 'Konum seçin',
  search: 'Bulgaristan'da yer arayın',
  useGps: 'Konumumu kullan (GPS)',
  cities: 'Şehirler',
  noResults: 'Böyle bir yer yok',
  abroadNote: 'Bulgaristan dışında konumunuz GPS'ten alınır.',
  close: 'Kapat',
},

date: {
  weekdaysShort: ['Paz', 'Pzt', 'Sal', 'Çar', 'Per', 'Cum', 'Cmt'],
  monthsShort: ['Oca', 'Şub', 'Mar', 'Nis', 'May', 'Haz', 'Tem', 'Ağu', 'Eyl', 'Eki', 'Kas', 'Ara'],
  monthsFull: ['Ocak', 'Şubat', 'Mart', 'Nisan', 'Mayıs', 'Haziran', 'Temmuz', 'Ağustos', 'Eylül',
'Ekim', 'Kasım', 'Aralık'],
},

hijriMonths: [
  'Muharrem',
  'Safer',
  'Rebiülevvel',
  'Rebiülahir',
  'Cemaziyelevvel',
  'Cemaziyelahir',
  'Recep',
  'Şaban',
  'Ramazan',
  'Şevval',
  'Zilkade',
  'Zilhicce',
],
};

```

index.ts src/i18n

- `resolveLang` – bg / tr / en.
- `pick(names)` – името на място или звук на текущия език; без турско име – английското.

src/i18n/index.ts

49 реда

```
import { getLocales, useLocales } from 'expo-localization';

import { bg, en, type Strings } from './strings';
import { tr } from './tr';

export type Lang = 'bg' | 'en' | 'tr';

const STRINGS: Record<Lang, Strings> = { bg, en, tr };

/**
 * Правилото (етап 10): езикът на телефона – български → български, турски → турски,
 * всеки друг – английски.
 */
export function resolveLang(languageCode: string | null | undefined): Lang {
  const code = languageCode?.toLowerCase();
  return code === 'bg' ? 'bg' : code === 'tr' ? 'tr' : 'en';
}

/** Име на два (или три) езика – местата, звуците. Турско липсва → английското. */
export interface Localized {
  bg: string;
  en: string;
  tr?: string;
}

export function pickName(names: Localized, lang: Lang): string {
  if (lang === 'bg') return names.bg;
  if (lang === 'tr') return names.tr ?? names.en;
  return names.en;
}

/**
 * Текущият език и текстовете за него.
 * `useLocales()` прерисува компонента, когато потребителят смени езика на телефона
 * (или езика само за това приложение – Android 13+ / iOS).
 */
export function useI18n(): { lang: Lang; t: Strings; pick: (names: Localized) => string } {
  const locales = useLocales();
  const lang = resolveLang(locales[0]?.languageCode);
  return { lang, t: STRINGS[lang], pick: (names) => pickName(names, lang) };
}

export type { Strings };

/** Същото без React – за известията, които се планират и във фонов режим. */
export function getI18n(): { lang: Lang; t: Strings; pick: (names: Localized) => string } {
  const lang = resolveLang(getLocales()[0]?.languageCode);
  return { lang, t: STRINGS[lang], pick: (names) => pickName(names, lang) };
}
```

tr.json locales

- Името под иконата – „Ezan“ – и текстът за разрешението за местоположение на iPhone.

locales/tr.json

9 реда

```
{
  "ios": {
    "CFBundleDisplayName": "Ezan",
    "NSLocationWhenInUseUsageDescription": "Ezan, bulunduğunuz yerin namaz vakitlerini göstermek için konumunuzu kullanır."
  },
  "android": {
    "app_name": "Ezan"
  }
}
```

app.json корен на проекта

- tr в locales и в supportedLocales на expo-localization.

app.json

173 реда

```
{
  "expo": {
    "name": "Adhan",
    "slug": "adhan",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "dark",
    "ios": {
      "supportsTablet": false,
      "bundleIdentifier": "com.ilkoadamov.adhan",
      "infoPlist": {
        "CFBundleAllowMixedLocalizations": true
      }
    },
    "android": {
      "adaptiveIcon": {
        "backgroundColor": "#0F1B2D",
        "foregroundImage": "./assets/android-icon-foreground.png",
        "backgroundImage": "./assets/android-icon-background.png",
        "monochromeImage": "./assets/android-icon-monochrome.png"
      },
      "predictiveBackGestureEnabled": false,
      "package": "com.ilkoadamov.adhan",
      "permissions": [
        "android.permission.SCHEDULE_EXACT_ALARM"
      ],
      "blockedPermissions": [
        "android.permission.READ_EXTERNAL_STORAGE",
        "android.permission.WRITE_EXTERNAL_STORAGE",
        "android.permission.RECORD_AUDIO"
      ]
    },
    "web": {
      "favicon": "./assets/favicon.png"
    },
    "plugins": [
```

```

"expo-router",
[
  "expo-splash-screen",
  {
    "backgroundColor": "#0F1B2D",
    "image": "./assets/splash-icon.png",
    "imageWidth": 160
  }
],
"expo-font",
[
  "expo-build-properties",
  {
    "android": {
      "cmakeVersion": "3.31.6"
    }
  }
],
[
  "expo-localization",
  {
    "supportedLocales": {
      "ios": [
        "en",
        "bg",
        "tr"
      ],
      "android": [
        "en",
        "bg",
        "tr"
      ]
    }
  }
],
[
  "expo-location",
  {
    "locationWhenInUsePermission": "Adhan uses your location to show the prayer times for the place
where you are.",
    "locationAlwaysAndWhenInUsePermission": false,
    "locationAlwaysPermission": false,
    "motionUsagePermission": false
  }
],
"expo-background-task",
[
  "expo-notifications",
  {
    "icon": "./assets/notification-icon.png",
    "color": "#D4A857",
    "sounds": [
      "./assets/sounds/ezan_chime.wav",
      "./assets/sounds/chime_soft.wav",
      "./assets/sounds/takbir_makkah.wav",
      "./assets/sounds/takbir_madinah.wav",
      "./assets/sounds/takbir_aqsa.wav",
      "./assets/sounds/takbir_istanbul.wav",
      "./assets/sounds/notify_airport.wav",
      "./assets/sounds/notify_popup.wav",
      "./assets/sounds/notify_right.wav",

```

```

        "./assets/sounds/notify_modern.wav",
        "./assets/sounds/notify_magical.wav",
        "./assets/sounds/notify_chime_bright.wav",
        "./assets/sounds/notify_doorbell.wav",
        "./assets/sounds/notify_celestial.wav",
        "./assets/sounds/notify_bell_ding.wav",
        "./assets/sounds/notify_crisp_ding.wav",
        "./assets/sounds/notify_shimmer.wav",
        "./assets/sounds/notify_smoke_alarm.wav",
        "./assets/sounds/notify_swoop.wav",
        "./assets/sounds/notify_retro.wav",
        "./assets/sounds/notify_robot.wav",
        "./assets/sounds/notify_three_beep.wav",
        "./assets/sounds/notify_machine_beep.wav",
        "./assets/sounds/notify_water_drop.wav",
        "./assets/sounds/notify_hang_up.wav",
        "./assets/sounds/notify_standard.wav"
    ]
}
],
[
    "expo-audio",
    {
        "microphonePermission": false,
        "recordAudioAndroid": false,
        "enableBackgroundPlayback": false,
        "enableBackgroundRecording": false
    }
],
"./plugins/withFullSounds",
[
    "expo-widgets",
    {
        "widgets": [
            {
                "name": "PrayerWidget",
                "displayName": "Adhan",
                "description": "The next prayer and the time left",
                "ios": {
                    "supportedFamilies": [
                        "systemSmall",
                        "systemMedium",
                        "accessoryCircular",
                        "accessoryRectangular",
                        "accessoryInline"
                    ]
                },
                "android": null
            },
            {
                "name": "PrayerArabicWidget",
                "displayName": "Adhan · Arabic",
                "description": "The next prayer in Arabic and the time left",
                "ios": {
                    "supportedFamilies": [
                        "accessoryCircular"
                    ]
                },
                "android": null
            }
        ]
    }
]

```

```

    }
  ]
},
"scheme": "adhan",
"backgroundColor": "#0F1B2D",
"experiments": {
  "typedRoutes": true
},
"locales": {
  "bg": "./locales/bg.json",
  "en": "./locales/en.json",
  "tr": "./locales/tr.json"
}
}
}

```

Стъпка 4

Градовете на турски

Турците в България казват „Kırcaali“, а не „Kardzhali“.

trNames.ts src/domain

- Традиционните турски имена на градовете от календара на Мюфтийството, областните центрове и няколко града с турско население. Селата остават на латиница – за тях няма надежден списък.

src/domain/trNames.ts

56 реда

```

/**
 * Турските (традиционните) имена на градовете в България – за турския език (етап 10).
 * Само градовете (областните центрове, градовете от календара на Мюфтийството и няколко
 * града с турско население); за останалите места остава латинското изписване.
 * Ключът е българското име.
 */
export const TR_PLACE_NAMES: Readonly<Record<string, string>> = {
  Айтос: 'Aydos',
  Балчик: 'Balçık',
  Благоевград: 'Yukarı Cuma',
  Бургас: 'Burgaz',
  Варна: 'Varna',
  'Велики Преслав': 'Preslav',
  'Велико Търново': 'Tırnova',
  'Гоце Делчев': 'Nevrokor',
  Добрич: 'Dobriç',
  Исперих: 'Kemallar',
  Каолиново: 'Bohçalar',
  Карлово: 'Kar'lova',
  Карнобат: 'Karinabad',
  Котел: 'Kazan',
  Крумовград: 'Koşukavak',
  Кубрат: 'Balbunar',
  Кърджали: 'Kırcaali',
  Ловеч: 'Lofça',
  Никопол: 'Niğbolu',
  'Нова Загора': 'Yeni Zağra',
  'Нови пазар': 'Yeni Pazar',
  Пазарджик: 'Pazarcık',
  Плевен: 'Plevne',
  Пловдив: 'Filibe',

```

```

Провадия: 'Pravadi',
Разград: 'Hazargrad',
Русе: 'Rusçuk',
Свищов: 'Ziştovi',
Силистра: 'Silistre',
Сливен: 'İslimiye',
Смолян: 'Paşmaklı',
София: 'Sofya',
'Стара Загора': 'Eski Zağra',
Търговище: 'Eski Cuma',
Харманли: 'Harmanlı',
Хасково: 'Hasköy',
Шумен: 'Şumnu',
Ямбол: 'Yanbolu',
Видин: 'Vidin',
Враца: 'İvraca',
Габрово: 'Gabrova',
Кюстендил: 'Köstendil',
Момчилград: 'Mestanlı',
Джебел: 'Cebel',
Ардино: 'Eğridere',
Омуртаг: 'Osmanpazarı',
Тутракан: 'Turtokan',
Дулово: 'Akkadınlar',
};

```

places.ts src/domain

- `placeToLocation` – турското име и уточнението „Kircaali ili“.
- `searchPlaces` – намира града и по турското име, със и без турските букви (ı, ş, ç, ğ, ö, ü).

src/domain/places.ts

277 реда

```

import data from '@data/places.json';

import { fastKm, insideBorder, nearBulgaria } from './geo';
import type { AppLocation } from './location';
import { muftiShiftFor } from './mufti';
import { normalizeForSearch, transliterate } from './translit';
import { TR_PLACE_NAMES } from './trNames';

/**
 * Всички ~6 700 населени места в България (OpenStreetMap, © OpenStreetMap contributors, ODbL).
 * Във файла: [име, ширина, дължина, индекс на област, индекс на община, вид].
 * Местностите („м. ...“), вилните зони и кварталите са махнати при подготовката
 * на файла – те не са населени места (оттам идваше „м. Детски град“ в София).
 */
export type PlaceKind = 0 | 1 | 2; // град, село, махала

export interface Place {
  index: number;
  name: string;
  lat: number;
  lon: number;
  oblast: string;
  obshtina: string;
  kind: PlaceKind;
}

type Row = [string, number, number, number, number, number];

```

```

const ROWS = data.places as Row[];

export const PLACES_SOURCE: string = data.source;
export const PLACES_COUNT = ROWS.length;

export function getPlace(index: number): Place {
  const [name, lat, lon, o, s, kind] = ROWS[index];
  return {
    index,
    name,
    lat,
    lon,
    oblast: data.oblasti[o],
    obshtina: data.obshtini[s],
    kind: kind as PlaceKind,
  };
}

/** Най-близката точка от списъка по права линия (за проверката „в България ли съм“). */
export function nearestPlace(lat: number, lon: number): { place: Place; km: number } {
  let best = 0;
  let bestKm = Infinity;
  const cosLat = Math.cos((lat * Math.PI) / 180);
  for (let i = 0; i < ROWS.length; i++) {
    const km = fastKm(lat, lon, ROWS[i][1], ROWS[i][2], cosLat);
    if (km < bestKm) {
      best = i;
      bestKm = km;
    }
  }
  return { place: getPlace(best), km: bestKm };
}

/**
 * Приблизителен радиус (км) на населеното място около точката му в OpenStreetMap.
 * Всяко място е само една точка (центърът), а големият град е широк километри:
 * в Младост си на 9 км от центъра на София и по-близо до точката на село Герман.
 */
const CITY_RADIUS_KM: Record<string, number> = {
  София: 12,
  Пловдив: 6,
  Варна: 7,
  Бургас: 6,
  Русе: 5,
  'Стара Загора': 4,
  Плевен: 4,
  Сливен: 3.5,
  Добрич: 3.5,
  Шумен: 3.5,
  Перник: 3.5,
  Хасково: 3.5,
  Ямбол: 3.5,
  Пазарджик: 3.5,
  Благоевград: 3.5,
  'Велико Търново': 3.5,
  Габрово: 4,
  Смолян: 4,
  Кърджали: 3,
  Кюстендил: 3,
  Враца: 3,
  Видин: 3,

```

```

Монтана: 3,
Ловеч: 3,
Силистра: 3,
Търговище: 3,
Разград: 3,
Казанлък: 3,
Асеновград: 3,
Дупница: 3,
'Горна Оряховица': 3,
Димитровград: 3,
};
const TOWN_RADIUS_KM = 2;
const VILLAGE_RADIUS_KM = 1;
const HAMLET_RADIUS_KM = 0.4;

function radiusKm(row: Row): number {
  if (row[5] === 0) return CITY_RADIUS_KM[row[0]] ?? TOWN_RADIUS_KM;
  return row[5] === 1 ? VILLAGE_RADIUS_KM : HAMLET_RADIUS_KM;
}

/**
 * В кое населено място съм по GPS.
 * 1. Ако съм в радиуса на едно или повече места – мястото, спрямо което съм
 * „най-навътре“ (разстояние / радиус). В Младост → София; в Кубратово,
 * което е в радиуса на София, но на 0,2 км от центъра си → Кубратово.
 * 2. Иначе – мястото с най-малко разстояние до края му (разстояние - радиус).
 */
export function placeAt(lat: number, lon: number): Place {
  let inside = -1;
  let insideRatio = Infinity;
  let outside = 0;
  let outsideGap = Infinity;
  const cosLat = Math.cos((lat * Math.PI) / 180);
  for (let i = 0; i < ROWS.length; i++) {
    const km = fastKm(lat, lon, ROWS[i][1], ROWS[i][2], cosLat);
    const r = radiusKm(ROWS[i]);
    if (km <= r) {
      if (km / r < insideRatio) {
        inside = i;
        insideRatio = km / r;
      }
    } else if (km - r < outsideGap) {
      outside = i;
      outsideGap = km - r;
    }
  }
  return getPlace(inside >= 0 ? inside : outside);
}

/**
 * В България ли е точката. Границата сама не стига: опростената брегова линия
 * „изразява“ някои плажове и крайбрежни селища (Балчик, Равда). Затова точка,
 * която е на под 1,5 км от българско населено място, също се брои за България.
 * 1,5 км е под разстоянието от Промахон (Гърция) до Кулата (2,1 км).
 */
export function isInBulgaria(lat: number, lon: number): boolean {
  if (!nearBulgaria(lat, lon)) return false; // Истанбул, Единбург... – без да обхождаме 6 700 места
  if (insideBorder(lat, lon)) return true;
  return nearestPlace(lat, lon).km < 1.5;
}

```

```

/** Името на мястото на двата езика и уточнение (община, област) - за различаване на еднакви имена. */
export function placeToLocation(place: Place): AppLocation {
  const cityLike = place.kind === 0;
  // турски: традиционното име на града (Кърджали → Kırcaali), иначе латиницата
  const trOr = (name: string) => TR_PLACE_NAMES[name] ?? transliterate(name);
  const trName = cityLike ? TR_PLACE_NAMES[place.name] : undefined;
  return {
    id: `bg-${place.index}`,
    names: { bg: place.name, en: transliterate(place.name), ...(trName ? { tr: trName } : {}) },
    detail: {
      bg: cityLike ? `обл. ${place.oblast}` : `общ. ${place.obshtina}, обл. ${place.oblast}`,
      en: cityLike
        ? `${transliterate(place.oblast)} Province`
        : `${transliterate(place.obshtina)}, ${transliterate(place.oblast)} Province`,
      tr: cityLike ? `${trOr(place.oblast)} ili` : `${trOr(place.obshtina)}, ${trOr(place.oblast)} ili`,
    },
    latitude: place.lat,
    longitude: place.lon,
    source: 'mufti',
    muftiShift: muftiShiftFor(place.lat, place.lon),
  };
}

/** Турските букви → латински: „Kırcaali“ се намира и като „kircaali“. */
function foldTurkish(text: string): string {
  return text
    .replace(/\\u0307/g, '') // „İ“.toLowerCase() → „i“
    .replace(/ı/g, 'i')
    .replace(/ş/g, 's')
    .replace(/ç/g, 'c')
    .replace(/ğ/g, 'g')
    .replace(/ö/g, 'o')
    .replace(/ü/g, 'u');
}

/** София-град от списъка - мястото по подразбиране, докато няма GPS или избор. */
export function defaultPlace(): Place {
  const i = ROWS.findIndex((r) => r[0] === 'София' && r[5] === 0);
  return getPlace(i);
}

// Индексът за търсене: 6 700 транслитерации - на компютъра ~50 ms, на телефона
// няколкостотин ms. Строи се на части, за да не блокира екрана (виж warmSearchIndex).
const searchIndex: { bg: string; lat: string; tr: string }[] = [];
const CHUNK = 500;

function buildMore(count: number): void {
  const end = Math.min(ROWS.length, searchIndex.length + count);
  for (let i = searchIndex.length; i < end; i++) {
    const trName = ROWS[i][5] === 0 ? TR_PLACE_NAMES[ROWS[i][0]] : undefined;
    searchIndex.push({
      bg: normalizeForSearch(ROWS[i][0]),
      lat: normalizeForSearch(transliterate(ROWS[i][0])),
      // турското име на града - и с, и без турските букви („kırcaali“, „kircaali“)
      tr: trName ? foldTurkish(normalizeForSearch(trName)) : '',
    });
  }
}

export function isSearchIndexReady(): boolean {
  return searchIndex.length === ROWS.length;
}

```

```

}

function index() {
  if (!isSearchIndexReady()) buildMore(ROWS.length); // търсене преди края - довършва наведнъж
  return searchIndex;
}

let warming: Promise<void> | null = null;
/**
 * Строи индекса на части от по 500 места, с пауза между тях - екранът и клавиатурата
 * остават живи. Вика се при отваряне на екрана за търсене; обещанието се изпълнява, когато е готов.
 */
export function warmSearchIndex(): Promise<void> {
  if (isSearchIndexReady()) return Promise.resolve();
  if (!warming) {
    warming = new Promise((resolve) => {
      const step = () => {
        buildMore(CHUNK);
        if (isSearchIndexReady()) resolve();
        else setTimeout(step, 0);
      };
      setTimeout(step, 0);
    });
  }
  return warming;
}

/**
 * Подредбата по азбука е направена в places.json при подготовката на данните.
 * Тук НЕ се ползва localeCompare: в Hermes (JS машината на телефона) всяко
 * localeCompare(..., 'bg') създава нов collator и е много бавно - сортирането на
 * 875 резултата за „ch“ замразяваше екрана за няколко секунди.
 * Затова при равенство се сравнява само индексът (= азбучният ред).
 */

let cities: Place[] | null = null;
/** Всички 256 града по азбучен ред - показват се, докато няма търсене. */
export function allCities(): Place[] {
  if (!cities) {
    cities = [];
    for (let i = 0; i < ROWS.length; i++) if (ROWS[i][5] === 0) cities.push(getPlace(i));
  }
  return cities;
}

/**
 * Търсене по име на кирилица или латиница („чеп“, „cher“).
 * Подредба: точно съвпадение → започва с → съдържа; при равенство - градове
 * преди села, после по азбука.
 */
export function searchPlaces(query: string, limit = 40): Place[] {
  const q = normalizeForSearch(query);
  if (q.length < 2) return [];
  const qt = foldTurkish(q);
  const idx = index();
  const hits: { i: number; rank: number }[] = [];
  for (let i = 0; i < idx.length; i++) {
    const { bg, lat, tr } = idx[i];
    let rank = -1;
    if (bg === q || lat === q || (tr && tr === qt)) rank = 0;
    else if (bg.startsWith(q) || lat.startsWith(q) || (tr && tr.startsWith(qt))) rank = 1;
  }

```

```

    else if (bg.includes(q) || lat.includes(q) || (tr && tr.includes(qt))) rank = 2;
    if (rank >= 0) hits.push({ i, rank: rank * 10 + ROWS[i][5] });
  }
  hits.sort((a, b) => a.rank - b.rank || a.i - b.i);
  return hits.slice(0, limit).map((h) => getPlace(h.i));
}

```

settings.ts src/store

- Миграция v3: вече избраното място в България получава турското име.

src/store/settings.ts

195 реда

```

import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

import {
  DEFAULT_HIGH_LAT_RULE,
  DEFAULT_LOCATION,
  DEFAULT_MADHAB,
  DEFAULT_METHOD,
} from '@config/defaults';
import {
  methodForCountry,
  type AsrMadhab,
  type CalculationMethodId,
  type HighLatRuleId,
  type MethodChoice,
} from '@domain/calc';
import type { AppLocation } from '@domain/location';
import { getPlace, PLACES_COUNT, placeToLocation } from '@domain/places';
import { countryOf } from '@domain/resolve';
import type { PrayerId } from '@domain/prayers';
import type { TimesOptions } from '@domain/times';

/** Граници на ръчните корекции. */
export const OFFSET_LIMIT = 15; // ±15 мин. за молитва
export const HIJRI_LIMIT = 2; // ±2 дни
/** Възможните напомняния преди молитвата (минути); 0 = без напомняне. */
export const REMINDER_CHOICES = [0, 5, 10, 15, 20, 30] as const;

const NO_OFFSETS: Record<PrayerId, number> = {
  fajr: 0,
  sunrise: 0,
  dhuhr: 0,
  asr: 0,
  maghrib: 0,
  isha: 0,
};

const clamp = (v: number, limit: number) => Math.max(-limit, Math.min(limit, v));

interface SettingsState {
  /** true - мястото идва от GPS; false - избрано ръчно. */
  autoLocation: boolean;
  /** Последното място от GPS (пази се, за да работи веднага при следващо отваряне и без GPS). */
  gpsLocation: AppLocation | null;
  /** Ръчно избраното място. */
  manualLocation: AppLocation | null;
}

```

```

/** 'auto' - според държавата (виж methodForCountry). */
method: MethodChoice;
madhab: AsrMadhab;
highLatRule: HighLatRuleId;
offsets: Record<PrayerId, number>;
hijriAdjust: number;
/** Напомняне X минути преди молитвата; 0 = без. */
reminderMinutes: number;
/** Вибрация при известие (Android; на iPhone решава телефонът). */
vibrate: boolean;

setAutoLocation: (on: boolean) => void;
setGpsLocation: (loc: AppLocation) => void;
chooseLocation: (loc: AppLocation) => void;
setMethod: (m: MethodChoice) => void;
setMadhab: (m: AsrMadhab) => void;
setHighLatRule: (r: HighLatRuleId) => void;
changeOffset: (id: PrayerId, delta: number) => void;
resetOffsets: () => void;
changeHijriAdjust: (delta: number) => void;
setReminderMinutes: (n: number) => void;
setVibrate: (on: boolean) => void;
}

export const useSettings = create<SettingsState>()(
  persist(
    (set) => ({
      autoLocation: true,
      gpsLocation: null,
      manualLocation: null,
      method: DEFAULT_METHOD,
      madhab: DEFAULT_MADHAB,
      highLatRule: DEFAULT_HIGH_LAT_RULE,
      offsets: NO_OFFSETS,
      hijriAdjust: 0,
      reminderMinutes: 0,
      vibrate: true,

      setAutoLocation: (on) => set({ autoLocation: on }),
      setGpsLocation: (loc) => set({ gpsLocation: loc }),
      // ръчният избор изключва GPS, за да не го „презapiше“ следващото определяне
      chooseLocation: (loc) => set({ manualLocation: loc, autoLocation: false }),
      setMethod: (method) => set({ method }),
      setMadhab: (madhab) => set({ madhab }),
      setHighLatRule: (highLatRule) => set({ highLatRule }),
      changeOffset: (id, delta) =>
        set((s) => ({ offsets: { ...s.offsets, [id]: clamp(s.offsets[id] + delta, OFFSET_LIMIT) } })),
      resetOffsets: () => set({ offsets: NO_OFFSETS }),
      changeHijriAdjust: (delta) =>
        set((s) => ({ hijriAdjust: clamp(s.hijriAdjust + delta, HIJRI_LIMIT) })),
      setReminderMinutes: (reminderMinutes) => set({ reminderMinutes }),
      setVibrate: (vibrate) => set({ vibrate }),
    }),
    {
      name: 'ezan.settings',
      // 2 (етап 5): методът по подразбиране е „Автоматично“ – старият „Диянет“ по подразбиране
      // става „Автоматично“ (за Турция и Европа пак е Диянет)
      // 3 (етап 10): турски език – запазените места в България получават и турското име
      version: 3,
      migrate: (persisted, version) => {

```

```

    const state = persisted as Partial<SettingsState>;
    if (version < 2 && state.method === 'Turkey') state.method = 'auto';
    if (version < 3) {
        state.manualLocation = withTurkishName(state.manualLocation);
        state.gpsLocation = withTurkishName(state.gpsLocation);
    }
    return state as SettingsState;
},
storage: createJSONStorage(() => AsyncStorage),
// новите полета (reminderMinutes, vibrate) липсват в старите записи - тогава важат стойностите по
подразбиране
partialize: ({
    autoLocation,
    gpsLocation,
    manualLocation,
    method,
    madhab,
    highLatRule,
    offsets,
    hijriAdjust,
    reminderMinutes,
    vibrate,
}) => ({
    reminderMinutes,
    vibrate,
    autoLocation,
    gpsLocation,
    manualLocation,
    method,
    madhab,
    highLatRule,
    offsets,
    hijriAdjust,
}),
},
),
);

/** Мястото, за което се показват часовете в момента. */
export function selectLocation(s: SettingsState): AppLocation {
    const chosen = s.autoLocation ? s.gpsLocation : s.manualLocation;
    return chosen ?? s.gpsLocation ?? DEFAULT_LOCATION;
}

type TimesParts = Pick<SettingsState, 'method' | 'madhab' | 'highLatRule' | 'offsets'> & { location:
AppLocation };

/** Опциите за часовете и техният „отпечатък“ `key` – по него се решава кога да се преизчисли. */
export function buildTimesOptions({ location, method, madhab, highLatRule, offsets }: TimesParts): {
    options: TimesOptions;
    key: string;
} {
    const options: TimesOptions = { location, method: resolveMethod(method, location), madhab, highLatRule,
offsets };
    const key = JSON.stringify([
        location.id,
        location.latitude,
        location.longitude,
        location.source,
        location.muftiShift,
        options.method,
    ]);

```

```

    madhab,
    highLatRule,
    offsets,
  });
  return { options, key };
}

/** „Автоматично“ → методът за държавата на мястото; иначе избраният. */
export function resolveMethod(method: MethodChoice, location: AppLocation): CalculationMethodId {
  return method === 'auto' ? methodForCountry(countryOf(location)) : method;
}

/** Същото, направо от състоянието – без React (ползва се и от известията във фонов режим). */
export function selectTimesOptions(s: SettingsState): { options: TimesOptions; key: string } {
  return buildTimesOptions({ ...s, location: selectLocation(s) });
}

/** Място в България, запазено преди етап 10: наново от списъка – с турското име. */
function withTurkishName<T extends AppLocation | null | undefined>(loc: T): T {
  if (!loc || !loc.id.startsWith('bg-')) return loc;
  const index = Number(loc.id.slice(3));
  if (!Number.isInteger(index) || index < 0 || index >= PLACES_COUNT) return loc;
  try {
    const fresh = placeToLocation(getPlace(index));
    return fresh.names.bg === loc.names.bg ? ({ ...loc, names: fresh.names, detail: fresh.detail } as T) :
loc;
  } catch {
    return loc;
  }
}

```

sounds.ts src/domain

- Турските имена на звуците: „Ezan · Mekke“, „Tekbir · Medine“, „Telefonun sesi“...

src/domain/sounds.ts

295 реда

```

import type { PrayerId } from './prayers';

/**
 * Звуците на Езан (етап 6).
 *
 * Два вида:
 * - 'full' – пълни звуци (целият езан, мелодии с всякаква дължина). Свири ги само истинската
 *   аларма на Android (етап 5). Файловете са в assets/sounds/full → res/raw.
 * - 'short' – кратки звуци до 30 сек. Свирият като звук на известие: на iPhone (там звукът на
 *   известие е най-много 30 сек.) и на Android без „Аларми и напомнания“.
 *   Файловете са в assets/sounds (wav) – в списъка на expo-notifications в app.json.
 *
 * Потребителят може да добави и свои звуци (виж src/services/sounds.ts): на Android – пълни,
 * на iPhone – кратки (до 30 сек.).
 */

export type SoundKind = 'full' | 'short';
export type SoundCategory = 'adhan' | 'takbir' | 'chime' | 'custom';

/** Специалният „файл“ на звука на телефона (системният звук за известия). */
export const SYSTEM_SOUND = 'system';
/** Своят звук за известия – до 10 сек.; по-дълъг не се добавя. */
export const NOTIFY_MAX_SEC = 10;

```

```

/** На iPhone звукът на известие може да е най-много 30 сек. - иначе iOS пуска стандартния. */
export const SHORT_MAX_SEC = 30;
/** Своят пълен звук - до 15 мин. (алармата спира предпазно след 15 мин.) и до 60 MB. */
export const FULL_MAX_SEC = 15 * 60;
export const MAX_UPLOAD_BYTES = 60 * 1024 * 1024;

export interface SoundDef {
  id: string;
  kind: SoundKind;
  category: SoundCategory;
  names: { bg: string; en: string; tr?: string };
  /**
   * Вграден: името на файла без разширение („adhan_makkah“ - res/raw на Android, а за кратките
   * и „takbir_makkah.wav“ в приложението за iPhone).
   * Свой: пълният път до файла (file:///...) - на Android; на iPhone - името в Library/Sounds.
   */
  file: string;
  /** Разширението на вградения кратък звук (за expo-notifications). */
  ext?: string;
  durationSec: number;
  /** Свой звук на iPhone, скъсен до 30 сек.: колко е бил. */
  originalSec?: number;
  custom?: boolean;
  /** Свой звук: за алармата или за известията. */
  use?: SoundUse;
}

/** Вграден звук за известията: assets/sounds/notify_<key>.wav. */
function notifySound(key: string, bg: string, en: string, tr: string, durationSec: number): SoundDef {
  return {
    id: `notify_${key}`,
    kind: 'short',
    category: 'chime',
    names: { bg, en, tr },
    file: `notify_${key}`,
    ext: 'wav',
    durationSec,
  };
}

/**
 * Вградените звуци. Записите на езан Мека, Медина и Ал-Акса са от колекцията на
 * PrayTimes.org, Истанбул - източникът да се уточни; изчистени, с изравнена сила (-16 LUFS);
 * откъсите с текбирите са отрязани на пауза (виж docs/PLAN.md - произходът им
 * трябва да се потвърди преди публикуване).
 */
export const BUILTIN_SOUNDS: readonly SoundDef[] = [
  // ---- пълни (само Android)
  {
    id: 'adhan_makkah',
    kind: 'full',
    category: 'adhan',
    names: { bg: 'Езан · Мека', en: 'Adhan · Makkah', tr: 'Ezan · Mekke' },
    file: 'adhan_makkah',
    durationSec: 201,
  },
  {
    id: 'adhan_madinah',
    kind: 'full',
    category: 'adhan',
  }

```

```

    names: { bg: 'Езан · Медина', en: 'Adhan · Madinah', tr: 'Ezan · Medine' },
    file: 'adhan_madinah',
    durationSec: 96,
  },
  {
    id: 'adhan_aqsa',
    kind: 'full',
    category: 'adhan',
    names: { bg: 'Езан · Ал-Акса', en: 'Adhan · Al-Aqsa', tr: 'Ezan · Mescid-i Aksa' },
    file: 'adhan_aqsa',
    durationSec: 225,
  },
  {
    id: 'adhan_istanbul',
    kind: 'full',
    category: 'adhan',
    names: { bg: 'Езан · Истанбул', en: 'Adhan · Istanbul', tr: 'Ezan · İstanbul' },
    file: 'adhan_istanbul',
    durationSec: 66,
  },
  // ---- кратки (iPhone и Android без точни аларми)
  {
    id: 'takbir_makkah',
    kind: 'short',
    category: 'takbir',
    names: { bg: 'Текбири · Мека', en: 'Takbir · Makkah', tr: 'Tekbir · Mekke' },
    file: 'takbir_makkah',
    ext: 'wav',
    durationSec: 30,
  },
  {
    id: 'takbir_madinah',
    kind: 'short',
    category: 'takbir',
    names: { bg: 'Текбири · Медина', en: 'Takbir · Madinah', tr: 'Tekbir · Medine' },
    file: 'takbir_madinah',
    ext: 'wav',
    durationSec: 25,
  },
  {
    id: 'takbir_aqsa',
    kind: 'short',
    category: 'takbir',
    names: { bg: 'Текбири · Ал-Акса', en: 'Takbir · Al-Aqsa', tr: 'Tekbir · Mescid-i Aksa' },
    file: 'takbir_aqsa',
    ext: 'wav',
    durationSec: 29,
  },
  {
    id: 'takbir_istanbul',
    kind: 'short',
    category: 'takbir',
    names: { bg: 'Текбири · Истанбул', en: 'Takbir · Istanbul', tr: 'Tekbir · İstanbul' },
    file: 'takbir_istanbul',
    ext: 'wav',
    durationSec: 30,
  },
  {
    id: 'chime',
    kind: 'short',
    category: 'chime',
  }

```

```

    names: { bg: 'Сигнал', en: 'Signal', tr: 'Sinyal' },
    file: 'ezan_chime',
    ext: 'wav',
    durationSec: 2,
  },
  {
    id: 'chime_soft',
    kind: 'short',
    category: 'chime',
    names: { bg: 'Тих звън', en: 'Soft chime', tr: 'Yumuşak çan' },
    file: 'chime_soft',
    ext: 'wav',
    durationSec: 4,
  },
  {
    // звукът за известия, избран в настройките на телефона
    id: 'system',
    kind: 'short',
    category: 'chime',
    names: { bg: 'Звукът на телефона', en: "The phone's sound", tr: 'Telefonun sesi' },
    file: SYSTEM_SOUND,
    durationSec: 0,
  },
  // ---- още звуци за известията (assets/sounds/notify_*.wav, до 10 сек.)
  notifySound('airport', 'Летище', 'Airport', 'Havalimanı', 3),
  notifySound('popup', 'Изскачане', 'Pop-up', 'Açılır', 1),
  notifySound('right', 'Правилно', 'Right', 'Doğru', 1),
  notifySound('modern', 'Модерен', 'Modern', 'Modern', 2),
  notifySound('magical', 'Вълшебен', 'Magical', 'Büyülü', 4),
  notifySound('chime_bright', 'Звънче', 'Chime', 'Çan', 3),
  notifySound('doorbell', 'Звънец на врата', 'Doorbell', 'Kapı zili', 2),
  notifySound('celestial', 'Небесен', 'Celestial', 'Göksel', 5),
  notifySound('bell_ding', 'Камбанка', 'Bell ding', 'Küçük çan', 1),
  notifySound('crisp_ding', 'Чист звън', 'Crisp ding', 'Net çan', 6),
  notifySound('shimmer', 'Блясък', 'Shimmer', 'Parıltı', 6),
  notifySound('smoke_alarm', 'Аларма за дим', 'Smoke alarm', 'Duman alarmı', 1),
  notifySound('swoop', 'Полъх', 'Swoop', 'Esinti', 1),
  notifySound('retro', 'Ретро', 'Retro', 'Retro', 1),
  notifySound('robot', 'Робот', 'Robot', 'Robot', 2),
  notifySound('three_beep', 'Три сигнала', 'Three beeps', 'Üç bip', 1),
  notifySound('machine_beep', 'Машинен сигнал', 'Machine beep', 'Makine bip', 1),
  notifySound('water_drop', 'Капка', 'Water drop', 'Su damlası', 1),
  notifySound('hang_up', 'Затваряне', 'Hang up', 'Kapatma', 1),
  notifySound('standard', 'Стандартен', 'Standard', 'Standart', 5),
];

export const DEFAULT_FULL = 'adhan_makkah';
export const DEFAULT_SHORT = 'takbir_makkah';
/** Звукът на известията и напомнията – по подразбиране звукът на телефона (решено 01.10.2026). */
export const DEFAULT_NOTIFY = 'system';

/** Молитвите, които имат аларма (изгревът е само известие). */
export const ALARM_PRAYERS: readonly PrayerId[] = ['fajr', 'dhuhr', 'asr', 'maghrib', 'isha'];

/** Свой звук, добавен от потребителя. */
export interface CustomSound {
  id: string;
  kind: SoundKind;
  name: string;
  /** Android: file:///... в паметта на приложението; iPhone: името на .caf файла в Library/Sounds. */
  file: string;
}

```

```

/** Колко свири (на iPhone - след скъсяването до 30 сек.). */
durationSec: number;
/** iPhone: колко е бил записът, ако е скъсен. */
originalSec?: number;
/** 'notify' - свой звук за известия (до 10 сек.); иначе - за алармата. */
use?: SoundUse;
}

export function customToDef(c: CustomSound): SoundDef {
  return {
    id: c.id,
    kind: c.kind,
    category: 'custom',
    names: { bg: c.name, en: c.name },
    file: c.file,
    ext: c.kind === 'short' ? 'caf' : undefined,
    durationSec: c.durationSec,
    originalSec: c.originalSec,
    custom: true,
    use: c.use ?? 'alarm',
  };
}

/** Всички звуци - вградените и своите. */
export function allSounds(custom: readonly CustomSound[]): SoundDef[] {
  return [...BUILTIN_SOUNDS, ...custom.map(customToDef)];
}

/** Звукът по id; ако го няма (изтрит свой звук) - резервният за вида. */
/**
 * За какво е звукът (решено 01.10.2026):
 * - 'alarm' - звукът на молитвата с камбанка „аларма“ (пълен или кратък резервен): езан,
 *           текбири и своите звуци (мелодии - само свои, вградени няма);
 * - 'notify' - звукът на известията и напомнянията: само кратките сигнали (звън).
 */
export type SoundUse = 'alarm' | 'notify';

export function fitsUse(s: SoundDef, use: SoundUse): boolean {
  if (s.custom) return (s.use ?? 'alarm') === use;
  return use === 'notify' ? s.category === 'chime' : s.category !== 'chime';
}

/** Звукът по id; ако го няма (изтрит свой звук) или не е за това - резервният. */
export function findSound(
  id: string | undefined,
  kind: SoundKind,
  custom: readonly CustomSound[],
  use: SoundUse = 'alarm',
): SoundDef {
  const all = allSounds(custom);
  const found = all.find((s) => s.id === id && s.kind === kind && fitsUse(s, use));
  if (found) return found;
  const fallback = use === 'notify' ? DEFAULT_NOTIFY : kind === 'full' ? DEFAULT_FULL : DEFAULT_SHORT;
  return all.find((s) => s.id === fallback!);
}

/** Името на файла за известие (expo-notifications): „takbir_makkah.wav“ / „custom_17.caf“. */
export function notificationFileName(s: SoundDef): string {
  if (s.id === 'system') return 'default'; // expo-notifications: звукът по подразбиране на телефона
  if (s.custom) return s.file; // на iPhone: името в Library/Sounds (с разширението)
  return `${s.file}.${s.ext ?? 'wav'}`;
}

```

```

}

/** „3:42“ */
export function formatDuration(sec: number): string {
  const s = Math.max(0, Math.round(sec));
  return `${Math.floor(s / 60)}:${String(s % 60).padStart(2, '0')}`;
}

/** Проверка на своя звук преди добавяне. null - става; иначе причината. */
export function validateUpload(
  kind: SoundKind,
  durationSec: number,
  bytes: number | undefined,
  use: SoundUse = 'alarm',
): 'tooLong' | 'tooBig' | 'empty' | null {
  if (!(durationSec > 0.3)) return 'empty';
  if (bytes !== undefined && bytes > MAX_UPLOAD_BYTES) return 'tooBig';
  if (use === 'notify' && durationSec > NOTIFY_MAX_SEC + 0.5) return 'tooLong';
  if (kind === 'short' && durationSec > SHORT_MAX_SEC + 0.5) return 'tooLong';
  if (kind === 'full' && durationSec > FULL_MAX_SEC) return 'tooLong';
  return null;
}

/** Името от файла: „Ezan_Bania-Bashi.mp3“ → „Ezan Bania-Bashi“. */
export function nameFromFile(fileName: string): string {
  const base = fileName.replace(/\.([^.] +$)/, '').replace(/[_]+/g, ' ').trim();
  return base.length > 40 ? `${base.slice(0, 39)}...` : base || 'Sound';
}

```

Стъпка 5

Android: widget-и и аларма

Native частта не взима текстовете от JavaScript само на две места – там са добавени турски ресурси.



„Günün vakitleri“ – прегледът в списъка с widget-и

ezan_widget_strings.xml modules/adhan-native/android/src/main/res/values-tr

- Имената и описанията на widget-ите в списъка на телефона; прегледите – в `drawable-tr-nodpi`.

modules/adhan-native/android/src/main/res/values-tr/ezan_widget_strings.xml

16 реда

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="ezan_widget_next_name">Sıradaki namaz</string>
    <string name="ezan_widget_next_desc">Sıradaki namaz ve kalan süre</string>
    <string name="ezan_widget_day_name">Günün vakitleri</string>
    <string name="ezan_widget_day_desc">Sıradaki namaz, kalan süre ve günün tüm vakitleri</string>
    <string name="ezan_widget_bar_name">Şerit</string>
    <string name="ezan_widget_bar_desc">Sıradaki namaz ve kalan süre tek satırda</string>
    <string name="ezan_widget_empty">Vakitleri görmek için Ezan'ı açın.</string>
    <string name="ezan_widget_sample_place">SOFYA</string>
    <string name="ezan_widget_sample_name">İkinci</string>
    <string name="ezan_widget_sample_at">16:33</string>
    <string name="ezan_widget_sample_at_long">saat 16:33</string>
    <string name="ezan_widget_sample_until">İkinci vaktine kalan</string>
    <string name="ezan_widget_sample_label">Sıradaki · </string>
</resources>
```

AlarmActivity.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- Датата на екрана „Аларма“ – и на турски („Pazartesi, 5 Ekim“).

modules/adhan-native/android/src/main/java/expo/modules/adhannative/AlarmActivity.kt

449 реда

```
package expo.modules.adhannative

import android.app.Activity
import android.content.Context
import android.content.Intent
import android.content.res.ColorStateList
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.LinearGradient
import android.graphics.Paint
import android.graphics.Path
import android.graphics.RectF
import android.graphics.Shader
import android.graphics.Typeface
import android.graphics.drawable.GradientDrawable
import android.graphics.drawable.RippleDrawable
import android.os.Build
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.text.TextUtils
import android.util.TypedValue
import android.view.Gravity
import android.view.KeyEvent
import android.view.View
import android.view.ViewGroup
import android.view.WindowManager
import android.widget.FrameLayout
import android.widget.LinearLayout
import android.widget.TextView
import java.text.SimpleDateFormat
```

```

import java.util.Date
import java.util.Locale
import kotlin.math.abs
import kotlin.math.cos
import kotlin.math.sin

/**
 * Екранът „Аларма“ на цял екран (по одобрения тоскир v2): час, дата, арабското име
 * на молитвата, „Време е за ...“, вълна с прогреса на езана, „Спри езана“ и „Заглуши звука“.
 * Показва се и върху заключения екран. Написан е изцяло на Kotlin, за да се отвори
 * веднага, без да се зарежда приложението.
 */
class AlarmActivity : Activity() {
    private val handler = Handler(Looper.getMainLooper())
    private var alarm: AlarmData? = null

    private lateinit var clock: TextView
    private lateinit var wave: WaveView
    private lateinit var bar: BarView
    private lateinit var timeText: TextView
    private lateinit var stopBtn: TextView
    private lateinit var muteBtn: TextView

    private val tick = object : Runnable {
        override fun run() {
            update()
            handler.postDelayed(this, 250)
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O_MR1) {
            setShowWhenLocked(true)
            setTurnScreenOn(true)
        } else {
            @Suppress("DEPRECATION")
            window.addFlags(WindowManager.LayoutParams.FLAG_SHOW_WHEN_LOCKED or
                WindowManager.LayoutParams.FLAG_TURN_SCREEN_ON)
        }
        window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)

        val a = AlarmService.current ?: AlarmData.fromJson(intent.getStringExtra(AlarmScheduler.EXTRA_ALARM))
        if (a == null) {
            finish()
            return
        }
        alarm = a
        setContentView(build(a))
        edgeToEdge(a)
    }

    override fun onNewIntent(intent: Intent) {
        super.onNewIntent(intent)
        // нова аларма, докато екранът е отворен – наново
        val a = AlarmService.current ?: AlarmData.fromJson(intent.getStringExtra(AlarmScheduler.EXTRA_ALARM))
        ?: return
        alarm = a
        setContentView(build(a))
        edgeToEdge(a)
    }
}

```

```

override fun onResume() {
    super.onResume()
    handler.post(tick)
}

override fun onPause() {
    super.onPause()
    handler.removeCallbacks(tick)
}

/** Бутоните за звука заглушават езана – както при будилника. */
override fun onKeyDown(keyCode: Int, event: KeyEvent?): Boolean {
    if (keyCode == KeyEvent.KEYCODE_VOLUME_UP || keyCode == KeyEvent.KEYCODE_VOLUME_DOWN) {
        if (AlarmService.state == AlarmService.State.PLAYING) {
            AlarmService.command(this, AlarmService.ACTION_MUTE)
            return true
        }
    }
    return super.onKeyDown(keyCode, event)
}

/* ----- обновяване */

private fun update() {
    val a = alarm ?: return
    val state = AlarmService.state
    // „Спри“ от известието или друга аларма – екранът се затваря
    if (state == AlarmService.State.IDLE || AlarmService.current?.id != a.id) {
        finish()
        return
    }
    clock.text = hm(System.currentTimeMillis())
    val playing = state == AlarmService.State.PLAYING
    val (pos, dur) = if (playing) AlarmService.progress() else Pair(AlarmService.lastDuration,
AlarmService.lastDuration)
    val fraction = if (dur > 0) pos.toFloat() / dur else 0f
    wave.playing = playing
    wave.fraction = fraction
    bar.fraction = fraction
    timeText.text = if (dur > 0) "${mmss(pos)} / ${mmss(dur)}" else ""
    if (playing) {
        stopBtn.text = a.labels.stop
        muteBtn.visibility = View.VISIBLE
    } else {
        stopBtn.text = a.labels.close
        muteBtn.visibility = View.INVISIBLE
    }
}

private fun stopAlarm() {
    AlarmService.command(this, AlarmService.ACTION_STOP)
    finish()
}

/* ----- изглед */

private fun dp(v: Float): Int = (v * resources.displayMetrics.density + 0.5f).toInt()

private fun font(name: String): Typeface =
    try {

```

```

        Typeface.createFromAsset(assets, "adhan_fonts/$name.ttf")
    } catch (e: Exception) {
        Typeface.DEFAULT_BOLD
    }

private fun text(
    value: String,
    size: Float,
    face: Typeface,
    color: Int = TEXT,
    unit: Int = TypedValue.COMPLEX_UNIT_SP,
): TextView =
    TextView(this).apply {
        text = value
        setTextSize(unit, size)
        typeface = face
        setTextColor(color)
        gravity = Gravity.CENTER
        includeFontPadding = false
    }

private fun build(a: AlarmData): View {
    val extraBold = font("manrope_extrabold")
    val bold = font("manrope_bold")
    val semi = font("manrope_semibold")
    val regular = font("manrope_regular")
    val amiri = font("amiri_bold")

    val root = FrameLayout(this)
    root.addView(BackgroundView(this, a.colors), FrameLayout.LayoutParams(MATCH, MATCH))

    val column = LinearLayout(this).apply {
        orientation = LinearLayout.VERTICAL
        gravity = Gravity.CENTER_HORIZONTAL
    }
    root.addView(column, FrameLayout.LayoutParams(MATCH, MATCH))

    // ---- горе: ЕЗАН, часът, датата
    val top = LinearLayout(this).apply {
        orientation = LinearLayout.VERTICAL
        gravity = Gravity.CENTER_HORIZONTAL
        setPadding(0, dp(26f), 0, 0)
    }
    top.addView(text(a.labels.app.uppercase(), 11f, extraBold, alpha(TEXT, 0.65f)).apply { letterSpacing = 0.16f })
    clock = text(hm(System.currentTimeMillis()), 72f, extraBold, TEXT, TypedValue.COMPLEX_UNIT_DIP).apply {
        letterSpacing = -0.03f
        setPadding(0, dp(2f), 0, dp(2f))
    }
    top.addView(clock)
    top.addView(text(dateLine(a), 14f, regular, alpha(TEXT, 0.75f)))
    column.addView(top, LinearLayout.LayoutParams(MATCH, WRAP))

    // ---- среда: арабското име, „Време е за ...“, мястото
    val mid = LinearLayout(this).apply {
        orientation = LinearLayout.VERTICAL
        gravity = Gravity.CENTER
        setPadding(dp(20f), 0, dp(20f), 0)
    }
    mid.addView(
        text(a.arabic, 64f, amiri, GOLD, TypedValue.COMPLEX_UNIT_DIP).apply {

```

```

        setShadowLayer(dp(15f).toFloat(), 0f, dp(4f).toFloat(), 0x59000000)
        includeFontPadding = true
    },
)
mid.addView(
    text(a.title, 21f, extraBold).apply {
        setPadding(0, dp(6f), 0, 0)
        maxLines = 2
        ellipsize = TextUtils.TruncateAt.END
    },
)
if (a.place.isNotEmpty()) {
    mid.addView(text(a.place, 13f, regular, alpha(TEXT, 0.7f)).apply { setPadding(0, dp(6f), 0, 0) })
}
column.addView(mid, LinearLayout.LayoutParams(MATCH, 0, 1f))

// ---- картата със звука: вълна, име и време, прогрес
val card = LinearLayout(this).apply {
    orientation = LinearLayout.VERTICAL
    setPadding(dp(16f), dp(16f), dp(16f), dp(14f))
    background = GradientDrawable().apply {
        cornerRadius = dp(22f).toFloat()
        setColor(Color.rgb(102, 6, 10, 20))
        setStroke(dp(1f), Color.rgb(20, 255, 255, 255))
    }
}
wave = WaveView(this)
card.addView(wave, LinearLayout.LayoutParams(MATCH, dp(38f)))
val trackRow = LinearLayout(this).apply {
    orientation = LinearLayout.HORIZONTAL
    setPadding(0, dp(12f), 0, dp(12f))
}
trackRow.addView(
    text(a.labels.soundName, 12.5f, semi).apply { gravity = Gravity.START },
    LinearLayout.LayoutParams(0, WRAP, 1f),
)
timeText = text("", 12.5f, semi, alpha(TEXT, 0.65f)).apply { gravity = Gravity.END }
trackRow.addView(timeText, LinearLayout.LayoutParams(WRAP, WRAP))
card.addView(trackRow, LinearLayout.LayoutParams(MATCH, WRAP))
bar = BarView(this)
card.addView(bar, LinearLayout.LayoutParams(MATCH, dp(4f)))
column.addView(
    card,
    LinearLayout.LayoutParams(MATCH, WRAP).apply {
        leftMargin = dp(22f)
        rightMargin = dp(22f)
    },
)

// ---- бутоните
val actions = LinearLayout(this).apply {
    orientation = LinearLayout.VERTICAL
    setPadding(dp(22f), dp(18f), dp(22f), dp(30f))
}
stopBtn = text(a.labels.stop, 17f, extraBold, GOLD_INK).apply {
    background = pill(GOLD, 0x33000000)
    isClickable = true
    setOnClickListener { stopAlarm() }
}
actions.addView(stopBtn, LinearLayout.LayoutParams(MATCH, dp(60f)))
muteBtn = text(a.labels.mute, 14.5f, bold, alpha(TEXT, 0.85f)).apply {

```

```

        background = pill(Color.TRANSPARENT, 0x33FFFFFF)
        isClickable = true
        setOnClickListener { AlarmService.command(this@AlarmActivity, AlarmService.ACTION_MUTE) }
    }
    actions.addView(muteBtn, LinearLayout.LayoutParams(MATCH, dp(46f)).apply { topMargin = dp(8f) })
    column.addView(actions, LinearLayout.LayoutParams(MATCH, WRAP))

    // отстъп за лентите на системата (часовник горе, навигация долу)
    column.setOnApplyWindowInsetsListener { v, insets ->
        @Suppress("DEPRECATION")
        v.setPadding(0, insets.systemWindowInsetTop, 0, insets.systemWindowInsetBottom)
        insets
    }
    return root
}

private fun pill(color: Int, ripple: Int): RippleDrawable {
    val shape = GradientDrawable().apply {
        cornerRadius = dp(999f).toFloat()
        setColor(color)
    }
    val mask = GradientDrawable().apply {
        cornerRadius = dp(999f).toFloat()
        setColor(Color.WHITE)
    }
    return RippleDrawable(ColorStateList.valueOf(ripple), shape, mask)
}

/** Фонът минава под лентите на системата; те са прозрачни. */
private fun edgeToEdge(a: AlarmData) {
    @Suppress("DEPRECATION")
    window.decorView.systemUiVisibility =
        View.SYSTEM_UI_FLAG_LAYOUT_STABLE or View.SYSTEM_UI_FLAG_LAYOUT_FULLSCREEN or
        View.SYSTEM_UI_FLAG_LAYOUT_HIDE_NAVIGATION
    @Suppress("DEPRECATION")
    window.statusBarColor = Color.TRANSPARENT
    @Suppress("DEPRECATION")
    window.navigationBarColor = Color.TRANSPARENT
    window.decorView.setBackgroundColor(a.colors[2])
}

/* ----- формати */

private fun hm(ms: Long): String = SimpleDateFormat("HH:mm", Locale.US).format(Date(ms))

private fun mmss(ms: Int): String {
    val s = ms / 1000
    return "${s / 60}:${(s % 60).toString().padStart(2, '0')}"
}

/** „Четвъртък, 1 октомври“ / „Thursday, 1 October“ */
private fun dateLine(a: AlarmData): String {
    val locale = when (a.lang) {
        "bg" -> Locale("bg")
        "tr" -> Locale("tr")
        else -> Locale.UK
    }
    val s = SimpleDateFormat("EEEE, d MMMM", locale).format(Date(a.at))
    return s.replaceFirstChar { it.titlecase(locale) }
}

```

```

companion object {
    private const val MATCH = ViewGroup.LayoutParams.MATCH_PARENT
    private const val WRAP = ViewGroup.LayoutParams.WRAP_CONTENT
    val TEXT = Color.parseColor("#F2EFE8")
    val GOLD = Color.parseColor("#D4A857")
    val GOLD_INK = Color.parseColor("#1A1408")

    fun alpha(color: Int, a: Float): Int = Color.argb((a * 255).toInt(), Color.red(color),
Color.green(color), Color.blue(color))
}

}

/** Градиентът на молитвата + геометричната шарка (8-лъчева звезда) със 7,5% злато. */
private class BackgroundView(context: Context, private val colors: IntArray) : View(context) {
    private val fill = Paint()
    private val line = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        style = Paint.Style.STROKE
        color = AlarmActivity.alpha(AlarmActivity.GOLD, 0.075f)
        strokeWidth = context.resources.displayMetrics.density
    }
    private val tile = Path()

    override fun onSizeChanged(w: Int, h: Int, oldw: Int, oldh: Int) {
        fill.shader = LinearGradient(0f, 0f, 0f, h.toFloat(), colors, floatArrayOf(0f, 0.55f, 1f),
Shader.TileMode.CLAMP)
        val u = context.resources.displayMetrics.density // 1 единица = 1 dp, плочката е 60 dp
        tile.reset()
        fun seg(x1: Float, y1: Float, x2: Float, y2: Float) {
            tile.moveTo(x1 * u, y1 * u)
            tile.lineTo(x2 * u, y2 * u)
        }
        tile.addRect(16 * u, 16 * u, 44 * u, 44 * u, Path.Direction.CW)
        tile.moveTo(30 * u, 10.2f * u)
        tile.lineTo(49.8f * u, 30 * u)
        tile.lineTo(30 * u, 49.8f * u)
        tile.lineTo(10.2f * u, 30 * u)
        tile.close()
        seg(0f, 0f, 16f, 16f)
        seg(60f, 0f, 44f, 16f)
        seg(0f, 60f, 16f, 44f)
        seg(60f, 60f, 44f, 44f)
        seg(30f, 0f, 30f, 10.2f)
        seg(30f, 49.8f, 30f, 60f)
        seg(0f, 30f, 10.2f, 30f)
        seg(49.8f, 30f, 60f, 30f)
        tile.addCircle(30 * u, 30 * u, 6 * u, Path.Direction.CW)
    }

    override fun onDraw(canvas: Canvas) {
        canvas.drawRect(0f, 0f, width.toFloat(), height.toFloat(), fill)
        val step = 60 * context.resources.displayMetrics.density
        var y = 0f
        while (y < height) {
            var x = 0f
            while (x < width) {
                canvas.save()
                canvas.translate(x, y)
                canvas.drawPath(tile, line)
                canvas.restore()
                x += step
            }
            y += step
        }
    }
}

```

```

        y += step
    }
}

/** Вълната: 30 златни черти, които „дишат“, докато звучи; изминалата част е по-ярка. */
private class WaveView(context: Context) : View(context) {
    var playing = true
    var fraction = 0f
    private val paint = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = AlarmActivity.GOLD }
    private val rect = RectF()
    private val heights = FloatArray(BARS) { w -> (0.35f + 0.65f * abs(sin(w * 1.7) * cos(w * 0.45))).toFloat() }

    override fun onDraw(canvas: Canvas) {
        val gap = 3 * context.resources.displayMetrics.density
        val barW = (width - gap * (BARS - 1)) / BARS
        val t = System.currentTimeMillis() / 1300.0 * 2 * Math.PI
        for (i in 0 until BARS) {
            val scale =
                if (playing) {
                    val phase = (sin(t + i * 0.55) + 1) / 2 // 0..1
                    (0.18f + (heights[i] - 0.18f) * phase.toFloat())
                } else {
                    0.12f
                }
            val h = height * scale
            val x = i * (barW + gap)
            rect.set(x, (height - h) / 2, x + barW, (height + h) / 2)
            val played = (i + 0.5f) / BARS <= fraction
            paint.alpha = if (!playing) 128 else if (played) 255 else 115
            canvas.drawRoundRect(rect, barW / 2, barW / 2, paint)
        }
        if (playing) postInvalidateOnAnimation()
    }

    companion object {
        private const val BARS = 30
    }
}

/** Лентата на прогреса под вълната. */
private class BarView(context: Context) : View(context) {
    var fraction = 0f
    set(v) {
        field = v.coerceIn(0f, 1f)
        invalidate()
    }
    private val track = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = Color.rgb(36, 255, 255) }
    private val fill = Paint(Paint.ANTI_ALIAS_FLAG).apply { color = AlarmActivity.TEXT }
    private val rect = RectF()

    override fun onDraw(canvas: Canvas) {
        val r = height / 2f
        rect.set(0f, 0f, width.toFloat(), height.toFloat())
        canvas.drawRoundRect(rect, r, r, track)
        rect.set(0f, 0f, width * fraction, height.toFloat())
        canvas.drawRoundRect(rect, r, r, fill)
    }
}

```

Достъпност

За хора с увеличен шрифт и за TalkBack (четец на екрана).

- **Голям шрифт** (Settings → Display → Font size): текстовете растат навсякъде. Само в редовете с точна височина – таблицата за месеца, списъкът за деня, кръгът с обратното броене, долната лента – растат до 1,2–1,35 пъти, за да не се застъпват.
- **TalkBack**: арабските имена се пропускат (иначе синтезаторът ги чете неразбираемо); всеки ден в таблицата за месеца се чете наведнъж – „Пн 5 окт: Фаджр 05:45, Изгрев 07:12...“; камбанките казват молитвата и състоянието.

typography.ts src/theme

- **FONT_SCALE** – докъде растат текстовете в плътните редове.

src/theme/typography.ts

25 реда

```
import type { TextStyle } from 'react-native';

/**
 * Имената съвпадат с ключовете, с които шрифтовете се зареждат в src/app/_layout.tsx.
 * В React Native всяка дебелина е отделно семейство, затова не ползваме fontWeight.
 */
export const fonts = {
  regular: 'Manrope_400Regular',
  medium: 'Manrope_500Medium',
  semibold: 'Manrope_600SemiBold',
  bold: 'Manrope_700Bold',
  extrabold: 'Manrope_800ExtraBold',
  arabic: 'Amiri_400Regular',
  arabicBold: 'Amiri_700Bold',
} as const;

/** Цифри с еднаква ширина, за да не „подскача“ обратното броене. */
export const tabularNums: TextStyle = { fontVariant: ['tabular-nums'] };

/**
 * Докъде растат текстовете при голям шрифт в настройките на телефона (достъпност, етап 10).
 * Навсякъде другаде текстът расте без ограничение; тук редовете са с точна височина
 * (таблицата за месеца, списъкът за деня, кръгът с обратното броене, долната лента).
 */
export const FONT_SCALE = { dense: 1.2, row: 1.35 } as const;
```

MonthTable.tsx src/components

- Целият ред за TalkBack.

src/components/MonthTable.tsx

122 реда

```
import { StyleSheet, Text, View } from 'react-native';

import { formatHM } from '@domain/format';
import { PRAYER_IDS } from '@domain/prayers';
import type { MonthDay } from '@domain/times';
import { useI18n } from '@i18n';
import { colors } from '@theme/colors';
```

```

import { FONT_SCALE, fonts, tabularNums } from '@theme/typography';

export const MONTH_ROW_HEIGHT = 42;
/** Височината на разделителя за нов месец по Хиджра (26 + 2 × 2 отстъп). */
export const HIJRI_SEPARATOR_HEIGHT = 30;
const DAY_COL = 50;

/** Заглавният ред: „Ден / Хиджра“ и имената на молитвите. */
export function MonthColumns() {
  const { t } = useI18n();
  return (
    <View style={[styles.row, styles.headRow]}>
      <View style={styles.dayCol}>
        <Text maxFontSizeMultiplier={FONT_SCALE.dense} style={[styles.head, styles.headLeft]}>
{t.month.dayColumn}</Text>
        <Text maxFontSizeMultiplier={FONT_SCALE.dense} style={styles.headHijri}>{t.month.hijriColumn}
</Text>
      </View>
      {PRAYER_IDS.map((id) => (
        <Text maxFontSizeMultiplier={FONT_SCALE.dense} key={id} style={styles.cell, styles.head}
numberOfLines={1} adjustsFontSizeToFit>
          {t.prayers[id]}
        </Text>
      ))}
    </View>
  );
}

/** Разделител на цялата ширина там, където започва нов месец по Хиджра: — Джамазиел-евел 1448 — */
export function HijriMonthSeparator({ label }: { label: string }) {
  return (
    <View style={styles.separator} accessibilityRole="header">
      <View style={styles.sepLine} />
      <Text maxFontSizeMultiplier={FONT_SCALE.dense} style={styles.sepText}>{label}</Text>
      <View style={styles.sepLine} />
    </View>
  );
}

interface RowProps {
  day: MonthDay;
  hijriDay: number;
  isToday: boolean;
}

/** Един ден: дата + ден от седмицата, ден по Хиджра и шестте часа. Петък е със златен ден от седмицата. */
export function MonthRow({ day, hijriDay, isToday }: RowProps) {
  const { t } = useI18n();
  const d = day.date;
  const isFriday = d.getDay() === 5;

  return (
    <View
      style={[styles.row, styles.dayRow, isToday && styles.today]}
      // за TalkBack/VoiceOver – целият ред наведнъж: „Пт 9 окт, петък: Фаджр 05:57, ...“
      accessible
      accessibilityLabel={` ${t.date.weekdaysShort[d.getDay()]} ${d.getDate()}
${t.date.monthsShort[d.getMonth()]}${
        isFriday ? `, ${t.month.friday}` : ''
      }${isToday ? `, ${t.month.today}` : ''}: ${day.times.map((p) => `${t.prayers[p.id]}
${formatHM(p.time)}').join(', ')} `}
    >

```

```

    >
    <View style={styles.dayCol}>
      <View style={styles.dateLine}>
        <Text maxFontSizeMultiplier={FONT_SCALE.dense} style={[[styles.dateNum, isToday && styles.gold]]}>
{d.getDate()}</Text>
        <Text maxFontSizeMultiplier={FONT_SCALE.dense} style={[[styles.weekday, isFriday && styles.gold]]}>
{t.date.weekdaysShort[d.getDay()]}</Text>
      </View>
      <Text maxFontSizeMultiplier={FONT_SCALE.dense} style={styles.hijriDay}>{hijriDay}</Text>
    </View>
    {day.times.map((p) => (
      <Text maxFontSizeMultiplier={FONT_SCALE.dense} key={p.id} style={[[styles.cell, styles.time, isToday
&& styles.timeToday]]}>
        {formatHM(p.time)}
      </Text>
    ))}
  </View>
);
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', paddingHorizontal: 8 },
  headRow: {
    height: 36,
    borderBottomWidth: StyleSheet.hairlineWidth,
    borderBottomColor: 'rgba(255,255,255,0.14)',
  },
  head: {
    fontFamily: fonts.bold,
    fontSize: 10.5,
    letterSpacing: 0.3,
    color: colors.muted,
    textAlign: 'center',
  },
  headLeft: { textAlign: 'left' },
  headHijri: { fontFamily: fonts.semibold, fontSize: 9, color: 'rgba(212,168,87,0.8)' },
  separator: {
    height: HIJRI_SEPARATOR_HEIGHT - 4,
    marginVertical: 2,
    flexDirection: 'row',
    alignItems: 'center',
    gap: 8,
    paddingHorizontal: 8,
  },
  sepLine: { flex: 1, height: 1, backgroundColor: 'rgba(212,168,87,0.45)' },
  sepText: { fontFamily: fonts.extrabold, fontSize: 11, letterSpacing: 0.3, color: colors.gold },
  dayRow: { height: MONTH_ROW_HEIGHT, borderRadius: 12 },
  today: {
    backgroundColor: colors.nowRow,
    borderWidth: 1,
    borderColor: colors.nowRowBorder,
  },
  dayCol: { width: DAY_COL },
  dateLine: { flexDirection: 'row', alignItems: 'baseline', gap: 4 },
  dateNum: { fontFamily: fonts.extrabold, fontSize: 14, color: colors.text, ...tabularNums },
  weekday: { fontFamily: fonts.semibold, fontSize: 10.5, color: colors.muted },
  hijriDay: { fontFamily: fonts.medium, fontSize: 10, color: 'rgba(212,168,87,0.75)', ...tabularNums },
  gold: { color: colors.gold },
  cell: { flex: 1, textAlign: 'center' },
  time: { fontFamily: fonts.semibold, fontSize: 13, color: colors.text, ...tabularNums },

```

```
timeToday: { fontFamily: fonts.extrabold, color: colors.gold },
});
```

PrayerRow.tsx src/components

- Арабското име – скрито от TalkBack.

src/components/PrayerRow.tsx

95 реда

```
import { StyleSheet, Text, View } from 'react-native';

import { formatHM } from '@/domain/format';
import { PRAYERS, type PrayerTime, type RowState } from '@/domain/prayers';
import { useI18n } from '@/i18n';
import type { AlertMode } from '@/store/alertPrefs';
import { colors } from '@/theme/colors';
import { FONT_SCALE, fonts, tabularNums } from '@/theme/typography';

import { BellButton } from './BellButton';

interface Props {
  prayer: PrayerTime;
  state: RowState;
  mode: AlertMode;
  onBellPress: () => void;
  muted?: boolean;
}

export function PrayerRow({ prayer, state, mode, onBellPress, muted }: Props) {
  const { t } = useI18n();
  const meta = PRAYERS[prayer.id];
  const name = t.prayers[prayer.id];
  const isNow = state === 'now';

  return (
    <View style={[styles.row, isNow && styles.now, state === 'past' && styles.past]}>
      <View style={styles.names}>
        <Text maxFontSizeMultiplier={FONT_SCALE.row} style={[styles.name, isNow && styles.nameNow]}>{name}</Text>
      </View>
      <Text
        maxFontSizeMultiplier={FONT_SCALE.row}
        style={styles.arabic}
        // арабското име - само за окото; TalkBack/VoiceOver го пропуска
        importantForAccessibility="no"
        accessibilityElementsHidden
      >
        {meta.arabic}
      </Text>
      <View style={styles.pill}>
        <Text maxFontSizeMultiplier={FONT_SCALE.row} style={styles.pillText}>{t.nowPill}</Text>
      </View>
    </View>
    <Text maxFontSizeMultiplier={FONT_SCALE.row} style={[styles.time, isNow && styles.timeNow]}>
      {formatHM(prayer.time)}</Text>
    <BellButton
      mode={mode}
      accessibilityLabel={t.a11y.bell(name, t.alert[mode])}
      onPress={onBellPress}>
```

```

        muted={muted}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  row: {
    flexDirection: 'row',
    alignItems: 'center',
    height: 46,
    paddingLeft: 14,
    paddingRight: 6,
    borderRadius: 15,
    gap: 12,
  },
  now: {
    backgroundColor: colors.nowRow,
    borderWidth: 1,
    borderColor: colors.nowRowBorder,
  },
  past: { opacity: 0.5 },
  names: { flex: 1, flexDirection: 'row', alignItems: 'center', gap: 8, minWidth: 0 },
  name: { fontFamily: fonts.semibold, fontSize: 15, color: colors.text },
  nameNow: { fontFamily: fonts.extrabold },
  arabic: {
    fontFamily: fonts.arabic,
    fontSize: 15,
    lineHeight: 22,
    color: 'rgba(242,239,232,0.5)',
  },
  pill: {
    backgroundColor: colors.gold,
    borderRadius: 999,
    paddingHorizontal: 7,
    paddingVertical: 2,
  },
  pillText: {
    fontFamily: fonts.extrabold,
    fontSize: 10,
    letterSpacing: 0.8,
    color: colors.goldInk,
  },
  time: { fontFamily: fonts.bold, fontSize: 16, color: colors.text, ...tabularNums },
  timeNow: { color: colors.gold },
});

```

Стъпка 7

Тестове

237 теста общо, 11 нови.

turkish.test.ts src/domain/__tests__

- Кой език по телефона; името без турски вариант → английското.
- Турските текстове имат същите ключове като българските и английските; молитвите и месеците; изреченията без наставки.

- Градовете: Kırcaali, „Kırcaali ili“; търсене по „kırcaali“, „Şumnu“, „İslimiye“, „Filibe“; widget-ът на турски.

src/domain/__tests__/turkish.test.ts

99 реда

```
import { pickName, resolveLang } from '@/i18n';
import { bg, en } from '@/i18n/strings';
import { tr } from '@/i18n/tr';
import { PHASE_GRADIENTS } from '@/theme/gradients';

import { BUILTIN_SOUNDS } from '../sounds';
import { defaultPlace, getPlace, placeToLocation, searchPlaces } from '../places';
import { locationFromCoords } from '../resolve';
import { widgetEntryAt } from '../widget';

/** Всички листа (текстове и функции) на обекта с текстовете - с пътя им. */
function leaves(o: unknown, path = ''): string[] {
  if (typeof o === 'string' || typeof o === 'function') return [path];
  if (Array.isArray(o)) return o.flatMap((x, i) => leaves(x, `${path}[${i}]`));
  if (o && typeof o === 'object') return Object.entries(o).flatMap(([k, v]) => leaves(v, path ?
`${path}.${k}` : k));
  return [path];
}

describe('турски език - кой език', () => {
  it('телефонът на турски → турски; български → български; другите → английски', () => {
    expect(resolveLang('tr')).toBe('tr');
    expect(resolveLang('TR')).toBe('tr');
    expect(resolveLang('bg')).toBe('bg');
    expect(resolveLang('de')).toBe('en');
    expect(resolveLang(undefined)).toBe('en');
  });

  it('име без турски вариант → английското', () => {
    expect(pickName({ bg: 'София', en: 'Sofia', tr: 'Sofya' }, 'tr')).toBe('Sofya');
    expect(pickName({ bg: 'Чепинци', en: 'Chepintsi' }, 'tr')).toBe('Chepintsi');
    expect(pickName({ bg: 'Чепинци', en: 'Chepintsi' }, 'bg')).toBe('Чепинци');
  });
});

describe('турски език - текстовете', () => {
  it('има същите ключове като българския и английския', () => {
    expect(leaves(tr).sort()).toEqual(leaves(bg).sort());
    expect(leaves(tr).sort()).toEqual(leaves(en).sort());
  });

  it('имената на молитвите и месеците - както в календара на Диянет', () => {
    expect(Object.values(tr.prayers)).toEqual(['İmsak', 'Güneş', 'Öğle', 'İkindi', 'Akşam', 'Yatsı']);
    expect(tr.hijriMonths[8]).toBe('Ramazan');
    expect(tr.date.weekdaysShort[5]).toBe('Cum');
  });

  it('изреченията с името на молитвата не искат турски наставки', () => {
    expect(tr.alarm.title(tr.prayers.asr)).toBe('İkindi vakti');
    expect(tr.widget.until(tr.prayers.maghrib)).toBe('Akşam vaktine kalan');
    expect(tr.notifications.reminderTitle(tr.prayers.isha, 10, '20:32')).toBe('Yatsı vaktine 10 dk - 20:32');
  });

  it('всички вградени звуци имат турско име', () => {
    for (const s of BUILTIN_SOUNDS) expect(s.names.tr).toBeTruthy();
  });
});
```

```
});

describe('турски език - местата', () => {
  it('градовете в България - с традиционното турско име', () => {
    const kardzhali = searchPlaces('Кърджали')[0];
    const loc = placeToLocation(kardzhali);
    expect(loc.names.tr).toBe('Kırcaali');
    expect(loc.detail?.tr).toBe('Kırcaali ili');
    expect(placeToLocation(defaultPlace()).names.tr).toBe('Sofya');
  });

  it('село - латиницата и турско уточнение за общината и областта', () => {
    const loc = locationFromCoords(41.4352, 24.869); // Чепинци
    expect(loc.names.tr).toBeUndefined();
    expect(loc.detail?.tr).toMatch(/ili$/);
  });

  it('търсенето намира града и по турското име - със и без турските букви', () => {
    expect(searchPlaces('Kırcaali')[0].name).toBe('Кърджали');
    expect(searchPlaces('kırcaali')[0].name).toBe('Кърджали');
    expect(searchPlaces('Şumnu')[0].name).toBe('Шумен');
    expect(searchPlaces('islimiye')[0].name).toBe('Сливен');
    expect(searchPlaces('Filibe')[0].name).toBe('Пловдив');
  });

  it('widget-ът на турски', () => {
    const now = new Date(2026, 9, 5, 15, 20, 15);
    const e = widgetEntryAt(now, {
      options: { location: locationFromCoords(42.6977, 23.3219), method: 'Turkey', madhab: 'shafi' },
      placeName: 'Sofya',
      texts: { prayers: tr.prayers, ...tr.hero, ...tr.widget },
      gradients: PHASE_GRADIENTS,
    });
    expect(e.place).toBe('SOFYA');
    expect(e.nextName).toBe('İkindi');
    expect(e.atText).toBe('saat 16:33');
    expect(e.untilText).toBe('İkindi vaktine kalan');
  });

  it('getPlace връща същото място по индекс (за миграцията на запазените места)', () => {
    const p = defaultPlace();
    expect(getPlace(p.index).name).toBe('София');
  });
});
```

Трябва да видиш: Tests: 237 passed, 237 total

npm test

Стъпка 8

Проверка на телефона

Samsung и Huawei:

- Settings → Apps → Adhan → Language → Türkçe: всички екрани на турски, името под иконата – „Ezan“.
- Konum seçin → „kırcaali“ → Kırcaali; на „Bugün“ – „Kırcaali“.
- Известие и аларма на турски („İkindi vakti“, „Ezanı durdur“), датата на екрана „Аларма“ – на турски.

- Widget-ите: прегледите и текстовете на турски.
- Settings → Display → Font size – най-големият: „Днес“ и „Месец“ не се разпадат.
- Settings → Accessibility → TalkBack: мини през „Днес“ и „Месец“.
- Върни езика на български – всичко се връща.

Преди публикуване: преводът и турските имена на градовете – да ги прегледа човек с турски като роден език.

Терминал

```
git add -A
git status
git commit -m "Stage 10: Turkish and accessibility"
git tag stage-10
git push
git push origin stage-10
```